



***Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.***

Temas Específicos para la preparación de la Oposición al Cuerpo Superior de Sistemas y Tecnologías de la Información de la Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

Tema 88. Análisis funcional de sistemas, casos de uso e historias de usuario. Metodologías de desarrollo de sistemas. Metodologías ágiles: Scrum y Kanban

AUTOR: Vanesa Mancebo Domínguez

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	4
2	ANÁLISIS FUNCIONAL DE SISTEMAS.....	5
2.1	ESTABLECIMIENTO DE REQUISITOS DE ANÁLISIS	6
2.1.1	CONCEPTO DE REQUISITO O REQUERIMIENTO	7
2.1.1.1	CATEGORIZACIÓN DE LOS REQUISITOS.....	7
2.1.1.2	CARACTERÍSTICAS DE LOS REQUISITOS	8
2.1.1.3	IDENTIFICACIÓN ÚNICA DE CADA REQUISITO	8
2.1.1.4	INFORMACIÓN A REGISTRAR DE CADA REQUISITO	8
2.1.1.5	EJEMPLOS DE REQUISITOS	9
2.1.2	ACTIVIDADES DEL ESTABLECIMIENTO DE REQUISITOS	10
2.1.3	DIFICULTADES EN EL ESTABLECIMIENTO DE REQUISITOS.....	10
2.1.4	TÉCNICAS PARA EL ESTABLECIMIENTO DE REQUISITOS	11
2.1.4.1	SESIONES JAD Y JRP	12
2.1.4.2	PROTOTIPOS.....	13
2.1.4.3	BRAINSTORMING	14
2.1.4.4	ENTREVISTAS	14
2.1.4.5	PIECES	16
2.1.4.6	CASOS DE USO.....	16
2.1.4.7	HISTORIAS DE USUARIO	19
2.1.5	VALIDACIÓN DE REQUERIMIENTOS	19
2.1.6	PROCEDIMIENTOS DE GESTIÓN DE REQUISITOS	21
2.1.6.1	GESTIÓN DE CAMBIOS EN LOS REQUISITOS	21
2.1.6.2	CONTROL DE VERSIONES	22
2.1.6.3	APROXIMACIÓN A LOS CAMBIOS DE REQUERIMIENTOS EN METODOLOGÍAS ÁGILES.....	23
2.1.6.4	TRAZABILIDAD DE REQUISITOS	24
2.1.7	EL ANÁLISIS DE REQUISITOS EN MÉTRICA VERSIÓN 3.....	24
3	METODOLOGÍAS DE DESARROLLO DE SISTEMAS	26
3.1	DEFINICIÓN DE METODOLOGÍA	26
3.1.1	PROCEDIMIENTOS	27
3.1.2	MÉTODOS O TÉCNICAS	27
3.1.3	HERRAMIENTAS CASE	28
3.1.4	SOPORTE DOCUMENTAL	28
3.1.5	REQUISITOS DE UNA METODOLOGÍA	29
3.1.6	CARACTERÍSTICAS DE LAS METODOLOGÍAS.....	29
3.1.7	METODOLOGÍA EN COMPARACIÓN CON CICLO DE VIDA.....	30
3.2	CLASIFICACIÓN DE LAS METODOLOGÍAS	31
3.2.1	METODOLOGÍAS ESTRUCTURADAS.....	31
3.2.1.1	METODOLOGÍAS ESTRUCTURADAS ORIENTADAS A PROCESOS	31
3.2.1.2	METODOLOGÍAS ESTRUCTURADAS ORIENTADAS A DATOS	32
3.2.2	METODOLOGÍAS ORIENTADAS A OBJETOS.....	34
3.2.2.1	UML.....	35
3.2.3	METODOLOGÍAS DE TIEMPO REAL.....	37
3.2.4	METODOLOGÍAS ÁGILES	37
3.2.4.1	PRINCIPIOS DEL MANIFIESTO ÁGIL	38
3.2.4.2	PRINCIPALES DIFERENCIAS ENTRE LAS METODOLOGÍAS CLÁSICAS Y LAS ÁGILES	40
3.2.4.3	PRINCIPALES METODOLOGÍAS ÁGILES UTILIZADAS	40

3.2.4.4	SCRUM	41
3.2.4.5	KANBAN.....	45
3.2.4.6	XP - EXTREME PROGRAMMING (PROGRAMACIÓN EXTREMA).....	46
3.2.4.7	CICLO DE DESARROLLO:	46
3.2.4.8	ROLES XP	46
3.2.4.9	CARACTERÍSTICAS MÁS SIGNIFICATIVAS DE LA PROGRAMACIÓN EXTREMA .	47
3.2.4.10	EL CICLO DE VIDA IDEAL DE XP	47
3.2.4.11	VALORES EN LA PROGRAMACIÓN EXTREMA (XP)	48
3.2.5	FEATURE DRIVEN DEVELOPMENT.....	48
3.2.5.1	EL PROCESO.....	48
3.2.5.2	ROLES Y RESPONSABILIDADES.....	49
3.2.6	TDD - TEST DRIVEN DEVELOPMENT	50
3.2.6.1	CICLO DE DESARROLLO CONDUCTIDO POR PRUEBAS	50

1 Introducción

Desde la creación de los primeros computadores, hacia principios de los años 40 del siglo XX, hasta la actualidad ha habido grandes cambios en todo lo relacionado con los programas que se ejecutan en los ordenadores. Hemos pasado de unos pequeños programas, desarrollados en código máquina, o como mucho en ensamblador, a grandes sistemas, formados por millones de líneas de código, los cuales permiten funcionalidades impensables hace no mucho tiempo. Hemos pasado de sistemas en que el usuario, desarrollador y operador era la misma persona, a sistemas en que estos perfiles están completamente diferenciados, con unos niveles de formación muy diferentes, muchas veces no relacionados con aspectos técnicos.

Este gran crecimiento en el tamaño de los sistemas y esta separación en los perfiles de los participantes llevan por supuesto emparejado una necesidad de disponer de métodos que permitan hacer de ello una labor de ingeniería.

Ya desde la llamada “crisis del software” se puso de manifiesto que el software era un producto cada vez más complejo que requería un proceso de ingeniería en su desarrollo para garantizar el éxito en el mismo. En este contexto, se partió de un modelo de prueba y error (Code and Fix) y se evolucionó a metodologías más robustas como la metodología en cascada que pretendía garantizar el éxito mediante la sistematización del proceso, sin embargo, este tipo de metodologías han sido consideradas como demasiado rígidas dando paso a las llamadas Metodologías Ágiles.

En el presente tema se tratará también otra de las grandes dificultades del desarrollo software: la especificación de la labor de qué debe abordar el software, se trata de un paso primordial que no está exento de complicaciones.

2 Análisis funcional de sistemas

La fase de Análisis de los Sistemas de Información, si descartamos las fases previas de planificación estratégica y estudios de viabilidad de los sistemas, es la primera del desarrollo de un sistema.

Según la bibliografía, a esta etapa en ocasiones se la denomina Análisis del Sistema de Información, mientras que en otras se la denomina Análisis Funcional de Sistemas queriendo enfatizar el hecho de que esta fase se basa en describir la funcionalidad que debe proporcionar el sistema.

Según Métrica v3, el Análisis del Sistema de Información tiene por objetivo la obtención de una especificación detallada del sistema que satisfaga las necesidades de información de los usuarios y sirva de base para el posterior diseño del mismo. Durante esta etapa se delimita el alcance del sistema, se genera el catálogo de requisitos generales y se describe el sistema mediante unos modelos iniciales. A la especificación de requisitos se la suele conocer como ERS o Especificación de Requisitos Software.

Existe un vacío entre el problema real (descripción del sistema) y la definición de un sistema que resuelva el problema real (modelo de diseño del sistema). Con el análisis se pretende pasar del dominio del problema a un modelo del sistema que sea implementable en una máquina. El Análisis hace de puente entre el problema y el diseño, identificando las necesidades y restricciones del problema y generando unas especificaciones (modelo de análisis) que sirvan de base para crear un diseño

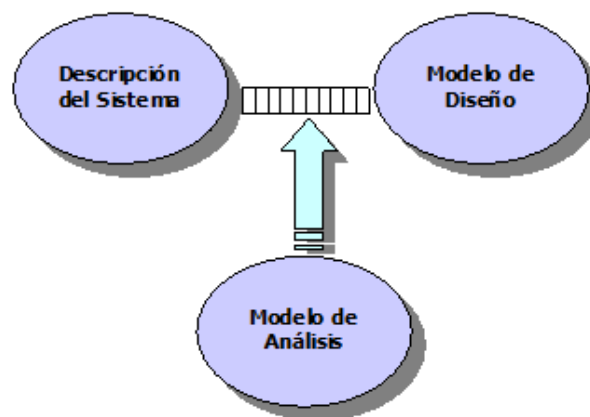


Figura 1 Modelo de Análisis

Según Pressman, el modelo de análisis debe cumplir tres objetivos:

- Describir lo que el cliente quiere.
- Establecer una base para la creación de un diseño de software.
- Definir un conjunto de requisitos que una vez el sistema construido se puedan validar.

El primer objetivo es vital, si el modelo de análisis no describe lo que el cliente quiere, como en las siguientes fases nos basaremos en el modelo de análisis, el sistema resultante tras todo el proceso de desarrollo no satisfará las necesidades del cliente.

El segundo punto, el establecimiento de una base para la creación de un diseño software, hace referencia a crear una especificación que defina, sin ninguna ambigüedad o clase de duda, el funcionamiento del sistema.

En cuanto al tercer objetivo, es muy común utilizar los requisitos del sistema como parte del contrato entre el desarrollador y el cliente. Si el conjunto de requisitos definido es muy vago o no son requisitos reales que puedan ser posteriormente validados, la relación contractual entre ambas partes puede verse resentida, ya que no habrá manera de demostrar, de forma fehaciente, que el sistema construido cumple o deja de cumplir con los requisitos.

La Ingeniería del Software ha desarrollado métodos orientados a la captura de las necesidades que tienen los usuarios, métodos que permiten facilitar el trabajo, pues rara vez los usuarios son capaces de explicarlos claramente. Mediante el estudio de forma iterativa del problema y aplicando diversos métodos y herramientas, se permite lograr una idea clara de qué espera el usuario obtener al final del ciclo de desarrollo. A lo largo de este tema se estudiarán técnicas que nos permitan obtener las necesidades reales del cliente y en posteriores temas se estudiarán técnicas para representar estas necesidades, nos centraremos en el Análisis de los Requisitos ya que el modelado del análisis se aborda en el tema 85.

2.1 Establecimiento de Requisitos de Análisis

Como dice Fred Brooks, *“La parte más difícil de construir un sistema es decidir qué construir. Ninguna parte del trabajo estropea tanto el sistema resultante si se hace mal. Ninguna parte es más difícil de rectificar después.”*

Con esta afirmación queda claro que un error mientras se decide qué se va a construir puede ser fatal para todo el proyecto. Una mala orientación del proyecto al principio del mismo puede hacer que éste siga un camino equivocado y se construya un sistema, que en el mejor de los casos no cubra todas las necesidades y expectativas del usuario, y que en el peor de los casos llegue a paralizar el desarrollo del proyecto y se cancele éste.

Como se ha visto anteriormente, el análisis de requisitos es una de las partes que componen la fase de análisis, en la que se obtiene del cliente las necesidades que éste tiene sobre un determinado problema. Existen numerosas definiciones de Gestión de Requisitos, la que propone el CMMI (Capability Maturity Model Integration) es "El objeto de la Gestión de Requisitos es establecer un entendimiento común entre el cliente y el proyecto de software respecto a los requisitos del cliente a abordar en el proyecto de software."

Para Thayer, "La ingeniería de requisitos proporciona el mecanismo apropiado para entender lo que el cliente quiere, analizar las necesidades, evaluar la factibilidad, negociar una solución razonable, especificar la solución sin ambigüedades, validar la especificación, y administrar los requisitos conforme se transforman en un sistema operacional".

De todas las definiciones existentes, el concepto que subyace bajo todas ellas es el mismo, es decir, que con el análisis de requisitos el analista comprenda el problema del cliente para así poder construir un sistema que responda satisfactoriamente a las necesidades del cliente.

Realmente, la Gestión de los Requisitos va más allá del análisis. Es tan importante el realizar un buen análisis de requisitos como posteriormente realizar una buena gestión de los cambios que puedan surgir en estos. No sirve de nada haber confeccionado un análisis de requisitos perfecto si durante el desarrollo del sistema estos van cambiando y no se van realizando las modificaciones adecuadas en el análisis de requisitos y documentos que se hayan derivado de estos.

Para garantizar que los requisitos son actualizados de forma coherente y consistente, y que se sigue un control en realizar dicha actividad, es necesario que se defina en el proyecto un Procedimiento de Gestión de Requisitos, para que todos los participantes conozcan cómo se deben gestionar.

La Gestión de Requisitos se encarga del tratamiento y control de las actualizaciones y cambios en los requisitos. La Gestión de Requisitos se realiza durante toda la vida del proyecto, no como el análisis de requisitos, que se hace al principio. Durante el desarrollo, especialmente en el mantenimiento, aparecerán nuevas necesidades que generarán nuevos requisitos, los cuales deberán integrarse con los requisitos existentes. Para poder realizar esta actividad es muy importante la trazabilidad, es decir, la capacidad para poder relacionar dos o más requisitos, y así poder analizar el impacto de un cambio.

La Gestión de Requisitos es una actividad esencial en un proyecto y que requiere recursos para su realización.

2.1.1 Concepto de Requisito o Requerimiento

En el análisis de requisitos, el elemento básico que se maneja durante las actividades es el requisito o también llamado requerimiento. Los requisitos se generan a partir de la interacción entre los usuarios y los desarrolladores, representando dichos requisitos las características del sistema a construir, es decir, las necesidades de los usuarios.

Según IEEE, un requisito es:

- Una condición o capacidad necesitada por un usuario para resolver un problema o alcanzar un objetivo.
- Una condición o capacidad que debe cumplir o poseer un sistema o un componente del mismo para satisfacer un contrato, un estándar, una especificación, u otro documento impuesto de una manera formal.
- Una representación documentada de una condición o capacidad tal como las expresadas en los dos puntos anteriores.

Desde el punto de vista del usuario, los requisitos son las condiciones o capacidades necesarias para que un usuario pueda resolver un problema o alcanzar un objetivo. Desde el punto de vista del equipo de desarrollo, los requisitos son las capacidades o condiciones que debe reunir un sistema para satisfacer un contrato, estándar o cualquier otro documento impuesto formalmente.

En resumen, los requisitos son las características que debe cumplir el sistema para que cubra las necesidades de los usuarios.

2.1.1.1 Categorización de los requisitos

Según Sommerville, los requisitos pueden clasificarse en requisitos a nivel de sistema y requisitos a nivel de software. Los requisitos a nivel de sistema hacen referencia no sólo al software, sino a todas las partes implicadas en el sistema que se está construyendo, como sistemas de comunicaciones, hardware, etc.

- **A nivel de sistema:**
 - **Requerimientos Funcionales Abstractos.** Son las funciones básicas que debe hacer el sistema a un nivel abstracto. Posteriormente, en cada uno de los subsistemas se concretan más.
 - **Propiedades del Sistema:** Propiedades no funcionales del sistema como por ejemplo disponibilidad, rendimiento, seguridad. Estas propiedades puede que no afecten a todos los subsistemas.
 - **Características que no debe mostrar el sistema.** En algunas ocasiones es importante especificar lo que el sistema no debe hacer. En un sistema de control de tráfico aéreo, se puede especificar que el sistema no debe especificar demasiada información al controlador.
- **A nivel del software:**
 - **Requerimientos Funcionales:** Son los requisitos que especifican la funcionalidad y el funcionamiento del software a construir.
 - **Requerimientos No Funcionales:** Son los requisitos que especifican aspectos adicionales que no corresponden con el funcionamiento del sistema. Estos requisitos no funcionales se pueden categorizar en:
 - Seguridad
 - Rendimiento
 - Entorno
 - Implantación
 - Disponibilidad del Sistema
 - Portabilidad
 - Fiabilidad
 - Eficiencia

- Ingeniería Humana
- facilidad de modificación
- Etc.

También según Sommerville, los requisitos se pueden categorizar como requisitos de usuario, los cuales dan una visión del sistema desde el punto de vista del usuario, o de sistema, los cuales dan una visión más detallada de la funcionalidad a proporcionar.

- **Requisitos de Usuario:** Declaraciones abstractas de los requerimientos del usuario final y el cliente. Se especifican normalmente como lenguaje natural o diagramas. Son los servicios que el sistema proporciona y las restricciones que debe cumplir.
- **Requisitos del Sistema:** Descripción más detallada de la funcionalidad a proporcionar. Establece con detalle funciones, servicios y restricciones operativas del sistema.

2.1.1.2 Características de los requisitos

Para que los requisitos sean de utilidad éstos deben cumplir una serie de características. Así, los requisitos deben ser:

- **Completo.** Los requisitos deben describir toda la información necesaria para construir el sistema.
- **Consistentes.** No debe haber inconsistencias o contradicciones entre distintos requisitos.
- **No Ambiguos.** Cada requisito debe dejar completamente claro, y sin ningún tipo de duda, la característica que describe del sistema.
- **Verificables.** Los requisitos del sistema deben poder ser verificados posteriormente, cuando el sistema esté construido, para poder probar que el sistema cumple con las necesidades del usuario.
- **Comprensibles.** Deben ser fáciles de entender, tanto por los usuarios como por los desarrolladores.
- **Fáciles de Modificar.** Deben poderse modificar fácilmente. Un cambio en un requisito no debe impactar en muchos cambios en el resto. El referenciar unos requisitos desde otros en vez de copiar el texto es una buena práctica.

2.1.1.3 Identificación única de cada requisito

Es vital para el proyecto que cada requisito se pueda identificar de forma única de entre el conjunto de requisitos. De esta forma, un requisito podrá hacer referencia a otros requisitos y también se podrán hacer matrices de trazabilidad que permitan relacionar en ellas cada una de las funcionalidades del sistema y a qué requisitos se deben dichas funcionalidades.

El código único puede estar basado en cualquier esquema de numeración que permita generar números únicos. Algunos ejemplos pueden ser:

- **Número secuencial.** Ej. 1, 2, 3, 4...
- **Esquema Jerárquico.** Ej. RF1.1.4, RF2.8.7, 5.3.5.4
- **Esquema Jerárquico y Subsistema.** Se puede definir un código para cada subsistema y después, dentro del subsistema usar un esquema jerárquico. Para proyectos grandes permite una primera partición de los requisitos en subsistemas de una forma sencilla e intuitiva. Ej. FAC.1.7.21.

2.1.1.4 Información a registrar de cada requisito

Será necesario que cada requisito esté descrito por al menos los siguientes datos:

- **Código Único.** Permitirá identificar al requisito.
- **Nombre Descriptivo del Requisito.** Será una frase que describa brevemente el requisito. Deberá permitir leer la frase y entender qué es lo que se describe en el requisito sin necesidad de leer la descripción completa.
- **Descripción.** Será la descripción detallada del requisito, explicando con el detalle que sea necesario las características del mismo. Se podrán incluir, si es necesario, diagramas, referencias a documentos del cliente, etc.

Adicionalmente es recomendable que se registren también los siguientes datos:

- Fecha de Creación
- Número de Versión
- Estado en el que se encuentra el requisito. Se indicará si el requisito está aceptado, rechazado, duplicado con otro, etc.
- Autor y/o organización que generó el requisito
- Prioridad o importancia del requisito en relación con el producto (especificado como alta, media o baja)
- Subsistema y / o funcionalidad a la que está asignado el requisito
- Versión o entrega del producto en que quedaría implantado el requisito
- Especificación del origen del requisito, la razón que lo respalda, si la hubiera, y dónde puede encontrarse esa información.
- Persona u organización que garantiza formalmente la implantación del requisito y método de verificación a utilizar.
- Criterio de pruebas de aceptación del requisito.
- Opcional u Obligatorio. Hay requisitos que son deseos del usuario pero que su implementación no es obligatoria, suelen ser candidatos a implementar en alguna versión posterior, otros, en cambio, son obligatorios y deberán implementarse ya en la primera versión.
- Beneficio, penalización, coste y riesgo que se espera tener por la implantación o no del requisito.
- Estabilidad del requisito. Especifica si el requisito tiene pocas probabilidades de cambiarse (requisito estable) o si por el contrario tiene muchas (volátil).

Si la gestión de requisitos se hace de forma manual, algunos de los datos anteriores son difíciles de mantener actualizados, como las fechas de actualización y los usuarios de actualización. Por el contrario, si se usa una herramienta de análisis de requisitos, ésta permitirá registrar todos los datos anteriores, además de permitir configurar otros datos que se consideren necesarios para un proyecto en particular.

2.1.1.5 Ejemplos de requisitos

A continuación se muestran algunos ejemplos de requisitos:

- **1:** El sistema deberá dar la respuesta a la consulta en un tiempo inferior a 1 segundo.
- **RF1:** El sistema deberá validar los datos de la entrada de la compra.
 - **RF1.1:** El nombre y apellidos se validará contra la base de datos corporativa de clientes.
 - **RF1.2:** La dirección se validará contra el sistema de callejero corporativo.
 - **RF1.3:** El importe de la compra no podrá ser inferior a 6€.
- **FAC.1.7.21:** El sistema ejecutará el ciclo de facturación diariamente con todos los clientes incluidos en dicho ciclo de facturación.

En el primer ejemplo se usa un esquema de numeración secuencial, mientras que en el segundo se usa un esquema jerárquico.

En el ejemplo se describe un requisito RF1, que es detallado posteriormente en otros requisitos de menor nivel, como RF1.1, RF1.2 y RF1.3.

En el último ejemplo se hace una división por subsistemas, al subsistema de Facturación se le identifica por el código FAC y dentro de éste se hace una subdivisión jerárquica como en el ejemplo anterior.

2.1.2 Actividades del Establecimiento de Requisitos

Aunque hay diferentes nomenclaturas que hacen referencia a las actividades incluidas en este punto, la secuencia básica de actividades es la siguiente:



Figura 2 Actividades de Establecimiento de Requisitos

A continuación se describe cada una de estas actividades:

1. **Educación de Requisitos:** En ella los analistas obtienen las necesidades del cliente a partir de todas las fuentes de información que tienen disponibles (documentación, entrevistas, estudio de los procesos de la organización, etc.).

Términos equivalentes usados por los ingenieros de software para esta actividad son Extracción de Requisitos, Identificación de Requisitos, Determinación de Requisitos, etc.

2. **Análisis de Requisitos:** En esta actividad se procede a trabajar sobre los requisitos educidos en el paso anterior. Se estudian los requisitos en busca de conflictos e incoherencias, implicaciones, información no obtenida y aspectos no resueltos. Después se clasifican, evalúa su viabilidad y se integran los nuevos requisitos con los ya existentes. El objetivo final es lograr una lista de requisitos que defina las necesidades del cliente.

3. **Representación de los Requisitos:** Actividad en la que se representan los requisitos de una o más formas, utilizando para ello diferentes técnicas como por ejemplo el lenguaje formal, el lenguaje natural, representaciones gráficas, etc.

Para la representación existen múltiples técnicas, las más usadas son, entre otras, los Diagramas de Flujo de Datos, Modelo Entidad Relación, Casos de Uso o Diagramas de Clases. Una vez que están los requisitos representados, es necesario que se reúnan los diversos participantes en el desarrollo para revisarlos y aprobarlos. El producto final con el que culmina esta fase es la Especificación de Requisitos Software, en donde está descrito con exactitud todo lo que el sistema debe hacer.

4. **Validación de Requisitos:** Actividad en la que se procede a definir una serie de criterios y técnicas que permitirán, cuando el sistema esté construido, comprobar que éste cumple los requisitos.

Las dos primeras actividades están en estrecha relación con el usuario o cliente, ya que se obtiene de ellos la información. Por el contrario, la tercera fase está más relacionada con el equipo de desarrollo, ya que los analistas, a partir de la información obtenida del cliente, se encargan de construir una serie de modelos, que posteriormente serán validados por el cliente, pero cuyo objetivo primordial es el de indicarle a los diseñadores qué se debe hacer.

Aunque estas cuatro actividades se han dibujado de forma secuencial en el diagrama, la realidad es que no existe una estricta secuencialidad entre ellas debido a que se llevan a cabo entrelazadamente y de forma repetitiva, hasta que las especificaciones del sistema están completamente confeccionadas.

2.1.3 Dificultades en el Establecimiento de Requisitos

Como se ha visto anteriormente, el proceso de obtención de requisitos es un proceso social, una comunicación entre personas, por lo que no está exento de problemas. Es importante conocer cuáles son estos problemas para así poder tratar de forma efectiva con ellos.

- El lenguaje de comunicación es en sí mismo ambiguo y se presta a la mala interpretación.

- Los usuarios tienen una vaga idea de lo que quieren. No saben expresar con claridad lo que quieren.
- Los usuarios y los desarrolladores tienen una visión diferente del sistema.
- Los usuarios no tienen una formación informática y no entienden todo lo que se les dice. Hay que tener cuidado con la terminología.

Los problemas de obtención de requisitos se pueden agrupar en tres categorías:

- **Problemas de Alcance:** Se debe especificar con precisión el alcance que va a tener el sistema, determinándose los límites y los objetivos del mismo. El alcance se puede definir en función de:
 - La Organización: Ofrecen una comprensión de la organización en la que se desarrolla el proyecto y del contexto en el que se desarrolla éste. Quien introduce las entradas al sistema, quien recibe los resultados, cambios en el modo de trabajo de la organización por introducir el nuevo sistema.
 - Del Entorno: Descripción del entorno en el que va a operar el sistema que se va a crear. Restricciones de hardware, software, si el sistema está dentro de otro más grande, etc.
 - Del Proyecto: Factores del contexto del proyecto que se va a desarrollar, como: conocimientos de los participantes, experiencia, jerarquía de personas, tiempo, coste, etc.
- **Problemas de Comprensión:** La obtención de requisitos es un proceso de comunicación entre personas que está expuesto a malentendidos y diferentes interpretaciones. Los problemas de comprensión se pueden dividir en:
 - Diferente formación de los participantes en el proyecto: informáticos, usuarios, control de calidad, etc.
 - Lenguaje demasiado formal o informal para ser entendido correctamente por todos. Un mismo término puede tener un significado general distinto al que tiene en un contexto en particular.
 - La información que se va obteniendo es necesario organizarla y transformarla, lo que la hace vulnerable a interpretaciones erróneas durante el procesado.
 - Visión diferente del proyecto por parte de cada uno de los participantes. La visión del desarrollador no es la misma que la del usuario o la de la alta dirección. Uno ve el proyecto como el trabajo a desarrollar mientras que el usuario lo ve como una herramienta que le ayudará en su trabajo y la alta dirección lo ve como una herramienta para lograr los objetivos.
- **Problemas de Volatilidad:** Las necesidades del usuario van cambiando a lo largo del proyecto, ya sea por razones externas al usuario o porque ha madurado las ideas y ha visto mejores formas de hacer algo. Las principales causas de volatilidad de son:
 - Las necesidades del usuario cambian.
 - Los requisitos son el resultado de integrar información de múltiples fuentes, algunas con intereses contrapuestos.
 - La complejidad de la organización puede hacer que a lo largo del proyecto se cambien objetivos, políticas, legislación, etc.

2.1.4 Técnicas para el establecimiento de Requisitos

Para poder lograr un buen análisis de requisitos se han creado una serie de técnicas que le facilitan al analista la labor de obtención y procesamiento de los requisitos. Todas estas técnicas han sido creadas para superar las dificultades inherentes al comportamiento humano.

Las técnicas de obtención de requisitos se pueden clasificar, en un primer nivel, en técnicas de alto nivel y técnicas de bajo nivel. Las técnicas de alto nivel permiten la obtención de requisitos generales, mientras que las técnicas de bajo nivel permiten obtener requisitos específicos de una parte del

sistema o de un usuario en particular. La correcta combinación de dichas técnicas permitirá al analista hacerse una idea clara del sistema que se debe construir y de las necesidades del usuario.

A continuación se muestran algunas técnicas de alto nivel y otras de bajo nivel. Algunas de dichas técnicas serán, por su interés, detalladas más adelante.

- **Técnicas de Alto Nivel**
 - Sesiones JAD Y JRP
 - Entorno de Bucles Adaptativo
 - Prototipos
 - Factores Críticos de Éxito
 - Historias de Usuario
- **Técnicas de Bajo Nivel**
 - Brainstorming
 - Entrevistas
 - PIECES
 - Casos de Uso
 - Análisis de Mercado

2.1.4.1 Sesiones JAD Y JRP

Las técnicas JAD y JRP son técnicas que promueven la cooperación entre los usuarios y los desarrolladores para lograr que ambas partes compartan una visión común. JAD y JRP son los acrónimos de:

JAD = Joint Application Design = Diseño Conjunto de Aplicaciones

JRP = Joint Requirements Planning = Planificación Conjunta de Requisitos

Tanto JAD como JRP siguen una estructura similar pero con participantes y contenidos a obtener diferentes. Mientras que JRP se usa para la planificación de requisitos, JAD se usa para el diseño conjunto de aplicaciones.

Ambas técnicas están formadas por el conjunto de pasos siguientes:

1. **Definición del Proyecto.** Se definen las características básicas del proyecto, es decir, propósito, alcance y objetivos del proyecto.
2. **Investigación.** El equipo se familiariza con el sistema, se investiga y documenta el flujo de trabajo, se reúnen especificaciones preliminares para los datos elementales, pantallas e informes.
3. **Preparación.** Se confecciona un documento de trabajo que sirva como base al documento JRP definitivo (documento de especificación de requisitos).
4. **Sesión JAD o JRP.** Partiendo del documento de trabajo del paso anterior, y a lo largo de entre 3 a 5 días de trabajo se definen las especificaciones finales del problema.
5. **Documento Final.** Se confecciona el documento final a partir de todos los formularios, diagramas, etc. generados durante la sesión.

La base de las técnicas JAD o JRP está en la confección del documento de partida confeccionado en el paso 3. Este documento será confeccionado por el equipo de desarrollo a partir de la información obtenida en el paso 1 de forma que cuando el equipo completo se reúna en el paso 4, desarrolladores y usuarios, se podrán conseguir un documento final en un plazo de 3 a 5 días.

La sesión JAD o JRP requiere que al menos haya personas para desempeñar los siguientes roles:

- Durante toda la sesión existirá un jefe o moderador JAD que se encargará de dirigir la sesión y de organizar la intervención de los diferentes participantes.
- Una persona se encargará de hacer un acta con todos los puntos importantes que vayan surgiendo en las reuniones.

Las sesiones JAD y JRP deberían durar media jornada. Hay que tener en cuenta que los usuarios normalmente tienen cometidos que cumplir en la organización, han sido asignados de forma temporal

al proyecto pero no se les descarga de su día a día. Por esta razón, si una sesión JAD o JRP dura medio día, permite a los usuarios que durante medio día hagan su trabajo diario o revisar la información generada el día anterior, y en el resto del día puedan participar en el desarrollo del sistema.

Para evitar que la sesión se desvíe y termine en largas discusiones, sobre temas de poca trascendencia, que no llevan a ninguna parte, es estrictamente necesario que se planifique ésta antes de empezar. Para cada tema se debe definir el tiempo que se quiere dedicar a dicho tema, y este tiempo no se puede exceder.

Para la ejecución del paso 4, la sesión propiamente dicha, es de gran utilidad el uso de ayudas audiovisuales como proyectores, pantallas gigantes, uso de herramientas informáticas para hacer diagramas, etc. La combinación de un proyector con una pantalla gigante y una herramienta de diagramado es especialmente útil, pues permite a todo el equipo seguir el razonamiento e ir viendo cómo se van confeccionando los diferentes diagramas.

Una vez que se ha confeccionado el documento final, consensuado por todos, es necesario que éste sea revisado por cada uno de los usuarios. Tras la sesión se enviará dicho documento a todos los participantes para que realicen las alegaciones que consideren necesarias al mismo.

Por último, para las sesiones JAD y JRP es de especial utilidad seguir el siguiente decálogo para que la reunión sea un éxito:

1. Compromiso con el Usuario.
2. Todos los participantes deben asistir a todas las sesiones.
3. En la sesión deben estar los participantes correctos para que puedan tomar decisiones.
4. Todos los participantes son iguales.
5. La preparación es tan importante como la sesión.
6. La documentación preparada antes de la sesión se considera una propuesta.
7. Celebrar la sesión en un lugar retirado para que sea más productivo.
8. Ser puntual en las sesiones.
9. Hacer una buena agenda y atenerse a ella.
10. Tratar de evitar la jerga técnica.

2.1.4.2 Prototipos

Los prototipos son sistemas software que sólo implementan una parte del sistema. Normalmente, los prototipos se emplean para obtener requisitos del usuario cuando éstos no están completamente claros. Es mucho más fácil que el usuario entienda y apruebe un sistema, que aunque es reducido también es operativo y puede interaccionar con él, a revisar una larga lista de texto con los requisitos que describen dicho sistema. Al fin y al cabo, como se suele decir, el prototipo permite salvar la situación siguiente: “No sé lo que quiero, pero lo sabré cuando lo vea”

Y en efecto, será mucho más fácil para un usuario saber si lo que quiere es lo que tiene delante y que puede interaccionar con él, que si es un taco de hojas con todos los requisitos escritos uno a uno.

El desarrollo de prototipos no siempre está recomendado, sino que debe usarse principalmente cuando se quiere probar un aspecto del sistema que no está claro, y el riesgo de cometer un error tiene un impacto considerable en el resultado del sistema. Hay que tener muy en cuenta que un prototipo cuesta una cantidad considerable de esfuerzo, y que sólo se justifica ese esfuerzo cuando el esfuerzo ahorrado es superior, para lo cual es muy recomendable el disponer de herramientas de desarrollo de prototipos, que permitan reducir los tiempos de desarrollo.

No se entrará en detalle en esta técnica pues existe en este temario un tema específico que trata sobre la técnica del prototipado.

2.1.4.3 Brainstorming

Esta técnica se usa principalmente para la generación de ideas en los casos en los que la generación de éstas no es obvia. Se trata de una técnica sencilla en la que se reúnen en grupo de 4 a 10 personas para generar ideas, sin restricciones, en un ambiente libre de críticas.

Uno de los principales puntos fuertes de la técnica es que ideas que en un principio pueden ser descabelladas tienen cabida. Primero se proporcionan las ideas y en una segunda vuelta se refinan.

El Brainstorming permite aportar a la obtención de requisitos los siguientes aspectos:

- **Genera múltiples puntos de vista de un problema.** Cada uno da su visión del problema, que no tiene porqué ser la misma. Una vez vistos todos los puntos de vista se puede atacar el problema de una forma más efectiva.
- **Formular un problema de distintas formas.** Cada participante puede ver el problema desde una óptica distinta, por lo que a la hora de formularlo lo hará de una forma distinta. Estas diferencias pueden aportar gran valor a la hora del estudio del problema, identificando los puntos clave del problema de cada participante.

La técnica de Brainstorming, para que sea efectiva, debe ser previamente preparada. Las fases por las que pasa la ejecución de la técnica son:

- **Fase de Preparación.**
 - Se identifican los participantes de la sesión (clientes, usuarios, analistas, etc.), se designa un líder de la sesión, se planifica y se busca una sala adecuada.
- **Fase de Generación.**
 - Se comienza exponiendo las ideas de forma libre, por turnos o espontáneamente. Las ideas se van apuntando en una pizarra que todos puedan ver.
- **Fase de Consolidación.**
 - Se revisan las ideas obtenidas en el paso anterior, se clarifican si no están claras, se rescriben si es necesario, se descartan las que no son utilizables, se discuten las restantes y se priorizan.

2.1.4.4 Entrevistas

Las entrevistas son una de las técnicas más importantes a la hora de obtener requisitos. Estas técnicas, funcionando independientemente, o como parte de otra técnica, permiten la obtención de información detallada.

Aunque las entrevistas pueden parecer una técnica fácil y trivial a primera vista, es necesario disponer de un conjunto de habilidades y planificarlas previamente para sacar el máximo partido de ellas.

Para poder realizar entrevistas, el entrevistador debe contar con las siguientes habilidades:

- Habilidades sociales para comunicarse con el interlocutor, empatía, elocuencia, etc.
- Capacidad para escuchar al entrevistado
- Conocimiento de las diversas tácticas de entrevista

2.1.4.4.1 Fases de una Entrevista

Las entrevistas tienen 4 fases durante su ejecución:

1. **Identificación de Candidatos:** Se identifican las personas a las cuales se va a entrevistar. Normalmente, el primero en ser entrevistado es el que ha autorizado o patrocina el proyecto.
2. **Preparación.** Se prepara la entrevista. Para ello se confeccionan las preguntas a realizar, se agrupan por temas y se les asigna un determinado tiempo a cada tema.
3. **Ejecución de la Entrevista.** La ejecución de la entrevista está formada por tres partes

diferenciadas:

- **Inicio:** Se abre la sesión, se le explican al entrevistado los motivos y las nomenclaturas de diagramas que se van a usar en la entrevista.
 - **Desarrollo de la Entrevista:** Se realizan las preguntas que se llevan planificadas dentro del tiempo asignado a cada una. Puede que a partir de una pregunta surjan nuevas para clarificar puntos que no quedan claros o puntos nuevos que han aparecido durante la misma.
 - **Finalización de la Entrevista:** La finalización es muy importante, pues en ella se resume, en 5 minutos todo lo visto en la sesión, para que ambas partes estén de acuerdo. De todas formas hay que indicar al entrevistado que la sesión se va escribir en papel y que se le enviará una copia al entrevistado, con lo que se ha hablado, para que lo revise.
4. **Actividades Posteriores a la Entrevista:** Se procederá a hacer un acta de la entrevista, la cual se enviará al entrevistado para que la revise y la confirme. La información que aparezca durante una entrevista debería ser corroborada con otras fuentes diferentes para asegurarse de que es fidedigna y no representa simplemente el punto de vista de un usuario, o incluso que se trata de una visión incorrecta que tiene el usuario.

2.1.4.4.2 Tipos de Preguntas en una Entrevista

Las preguntas que se realizan durante una entrevista pueden ser de los diferentes tipos:

- **Generales:** Se emplean para establecer el contexto del sistema. Serán las primeras en usarse o cuando se quiera entrar en un nuevo área diferente del que se estaba discutiendo. Son ejemplos: “¿Qué se espera del sistema?” “¿Cuáles son sus cometidos?”
- **Abiertas:** Este tipo de preguntas da pie a que el entrevistado hable y se explique, pudiéndose de dicha explicación obtener nueva información o clarificar la ya existente. Estas preguntas permiten obtener mucha información, aunque hay que saber controlar al interlocutor para que no se desvíe del tema principal. Ejemplo: “¿Dígame qué tiene que hacer cuando se presenta dicha situación?”
- **Cerradas:** Este tipo de preguntas proporciona detalles sobre un determinado aspecto. En estas preguntas, dependiendo de cómo se hagan, pueden causar que se lleve al entrevistado a corroborar nuestro punto de vista. Ejemplo: “¿qué fórmula se usa para calcular el índice de retraso?”

Si en vez de esta pregunta, se hace una pregunta como: “y el índice de retraso lo calcula como: ... ¿no?” puede que llevemos al entrevistado a corroborar nuestro punto de vista, que puede que no sea el correcto o sea una visión parcial de la situación.

- **Preguntas que Elevan el Nivel.** Son preguntas que se realizan para cambiar el nivel de abstracción de la misma. Así por ejemplo, si estamos analizando un tema con mucho detalle y queremos reducir el nivel de detalle podemos usar una pregunta como: “¿Cuál es el objetivo de eso?”

De esta forma pasaremos a ver el proceso como una caja negra y el entrevistado pasará a detallar los objetivos del mismo.

- **Preguntas de Contexto.** Este tipo de preguntas permite cambiar el contexto del que se está hablando. Son útiles para pasar de un tema a otro que no tiene relación. Es muy importante que el entrevistado se dé cuenta de que se ha hecho el cambio de contexto, para que se adapte al nuevo. Se debe evitar cambiar mucho de contexto, y cuando se haga dejar bien claro cuál es el nuevo contexto, para no confundir y desorientar al entrevistado.

2.1.4.5 PIECES

PIECES es una técnica que define seis categorías de aspectos que el analista debe estudiar con los usuarios. El nombre PIECES procede de un acrónimo formado por dichas 6 categorías, que son:

- Rendimiento (Performance)
- Información y Datos (Information)
- Economía (Economy)
- Control (Control)
- Eficiencia (Efficiency)
- Servicios (Services)

PIECES define un conjunto de preguntas estándar para cada una de las categorías, orientadas a la obtención de requisitos.

Algunos de los aspectos que PIECES evalúa para cada una de las seis categorías son:

- **Rendimiento** (Performance)
 - Throughput (Número de sucesos por unidad de tiempo)
 - Tiempo de Respuesta
- **Información y Datos** (Information)
 - Datos útiles para la toma de decisión
 - Acceso a la clase correcta de información, en el momento adecuado y cantidad justa
- **Economía** (Economy)
 - Nivel de Servicio a ofrecer
 - Exceso de Capacidad para soportar picos
- **Control** (Control)
 - Seguridad
 - Auditoría
- **Eficiencia** (Efficiency)
 - Aprovechamiento de los recursos disponibles
- **Servicios** (Services)
 - A los usuarios
 - A los clientes
 - A otros sistemas

2.1.4.6 Casos de Uso

Un caso de uso muestra la funcionalidad que el programa debe proporcionar a un usuario, se escriben haciendo hincapié en la secuencia de acciones realizadas por el sistema para dar respuesta a un usuario.

Tal y como se especifica en la metodología Métrica, los casos de uso permiten capturar los requisitos funcionales del sistema y expresarlos desde el punto de vista del usuario.

Aunque se trata de una técnica independiente del tipo de metodología y que puede aplicarse tanto a desarrollos estructurados como orientados a objetos, tiene especial relevancia en las Metodologías Orientadas a Objetos: PUDES se dice que está dirigido por los Casos de Uso, por su parte Métrica v3 considera esta técnica como opcional para el desarrollo estructurado mientras que la marca como obligatoria para el desarrollo orientado a objetos.

2.1.4.6.1 Especificación de los casos de uso

La especificación del caso de uso es la descripción textual de la funcionalidad soportada por ese caso de uso. En el caso de metodologías orientadas a objetos lo más habitual es refinar la descripción de los casos de uso a medida que se avanza en las fases del proyecto, de manera que se parte de una

descripción de alto nivel para detallar los requerimientos y se obtiene una descripción mucho más detallada en la fase de Diseño

Los casos de uso pueden clasificarse en:

- Esenciales: Cuando se describen de manera abstracta, independiente de la tecnología.
- Reales: Cuando incluyen más detalle de la solución específica y entran en detalles de la interfaz.

Durante la fase de Análisis Funcional, los casos de uso serán siempre esenciales.

Para la especificación de casos de uso se puede incluir la siguiente información:

- Nombre del Caso de Uso
- Identificador: identificador único para poder referenciarlo.
- Actores: Lista de los usuarios o sistemas involucrados en el caso de uso.
- Visión General: Descripción de alto nivel.
- Tipo: Indica si es primario, secundario u opcional (según la importancia).
- Referencias: otros casos de uso relacionados.
- Curso Típico de eventos: Interacción entre Actores y el Sistema ordenado en el escenario más habitual en el que el caso de uso se completa de manera normal
- Curso Alternativo de eventos: Alternativas y excepciones. Se trata de caminos alternativos por los que discurrirá el caso de uso dependiendo de condiciones en su realización (por ejemplo en caso de ocurrir algún error).

2.1.4.6.2 Diagrama de Casos de Uso

Los casos de uso suelen representarse en un Diagrama de Casos de Uso.

El objetivo de este diagrama es delimitar el ámbito del sistema y ofrecer una visión de alto nivel de la funcionalidad que debe proporcionar éste, resaltando los actores con las que interactúa y qué funcionalidad está disponible para actor.

Los elementos principales de este diagrama son los siguientes:

- Actores: Un actor es algo o alguien que se encuentra fuera del sistema y que interactúa con el sistema, en caso de ser un usuario representará un rol, no un usuario específico. Un ejemplo de actor sería el ciudadano.
Se representan mediante una figura de *hombre de palo*:

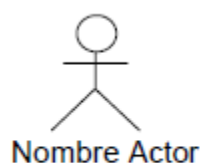


Figura 3 Notación Casos de Uso: Actores

- Caso de Uso: Representa la funcionalidad tal y como se explicó en el apartado anterior. Se representa con una elipse con el nombre del caso de uso en su interior o bajo la elipse:

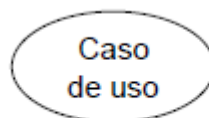


Figura 4 Notación Casos de Uso: Caso de Uso

- Relación entre casos de uso:
 - o **Comunica:** Es la relación entre un actor y un caso de uso, que denota la participación del actor en dicho caso de uso.
Se representa con una línea que une el actor o actores con el caso de uso.

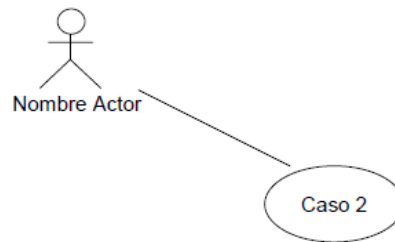


Figura 5 Notación Casos de Uso: Comunicación caso de uso y Actor

- o **Usa (incluye):** Relación de dependencia entre dos casos de uso que denota la inclusión del comportamiento de un escenario en otro. Se usa cuando se quiere reflejar un comportamiento común en varios casos de uso.
Se representa con una flecha en la que se incluye la palabra <<usa>> o <<includes>> y que apunta al caso utilizado desde el caso que lo utiliza.

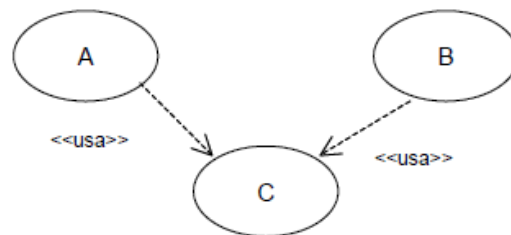


Figura 6 Notación Casos de Uso: Comunicación <<usa>>

En ese ejemplo tanto el caso de uso A como el B usan el caso de uso C.

- o **Extiende:** Relación de dependencia entre dos casos de uso en el que uno es una especialización del otro, existiendo en este caso una extensión de la funcionalidad. La principal diferencia con la relación de inclusión es que en este caso el caso de uso que extiende al otro se ejecutará sólo en algunos casos.
Se representa con una flecha en la que se incluye la palabra <<extiende>> o <<extends>>, la flecha apunta desde el caso de uso extendido al caso de uso de extensión.



Figura 7 Notación Casos de Uso: Comunicación <<extiende>>

En ese ejemplo, B extiende a A, es decir: cuando se ejecuta el caso de uso A, en función de alguna condición, puede ejecutarse el caso de uso B.

2.1.4.7 Historias de usuario

Las historias de usuario son un mecanismo utilizado muy frecuentemente en Metodologías Ágiles para documentar los requisitos. Se trata de un mecanismo mucho más reducido que los casos de uso. Según Mike Cohn, deberían seguir el siguiente formato:

“Como (rol) quiero (algo) de manera que obtenga (beneficio)”

2.1.4.7.1 Características de las historias de usuario (INVEST):

Bil Wake inventó el método INVEST para crear historias de usuario. Se trata de un acrónimo de las características que deben cumplir las historias de usuario

- Independientes: deben tener independencia entre sí, eso significa que podrían implementarse en cualquier orden.
- Negociables: la discusión con los usuarios debe permitir esclarecer su alcance y éste debe dejarse explícito bajo la forma de pruebas de validación.
- Valiosa: la funcionalidad proporciona valor para los clientes o los usuarios del software.
- Estimable: debe ser posible estimar el esfuerzo que requerirá construirla.
- Pequeña: las historias deberían construirse en poco tiempo. Se tienen que poder construir muchas historias en una iteración.
- Testeable (verificable): debe ser posible comprobar que está implementada.

Otras características de las historias de usuario:

- Las deberían escribir los stakeholders del proyecto, no los desarrolladores.
- Deben estar escritas en alguna herramienta sencilla de utilizar. Es común utilizar tarjetas para representarlas, en estas tarjetas se incluye la descripción, la prioridad y la información de estimación.
- La información de estimación indica el esfuerzo requerido para implementar ese requisito, es habitual expresarla en función de los llamados “puntos-historia”, los puntos-historia se convierten en horas aplicando un factor de conversión.
- La prioridad la debe indicar el product owner o los stakeholders. Esta información se utilizará para priorizar los requisitos a implementar en cada iteración.
- Pueden incluir tanto requisitos funcionales como no funcionales.
- Frecuentemente se utiliza un identificador para cada historia de usuario de manera que sea más sencillo hacer referencia a ella.

2.1.4.7.2 Historias de usuario épicas

Se denominan épicas a las historias de usuario demasiado grandes. Normalmente tienen que ver con requisitos poco priorizados y que por tanto no han sido detalladas, a medida que van a ser implementadas se irán detallando y separando en varias historias de usuario. Este término se utiliza también para agrupar historias de usuario relacionadas con una funcionalidad particular, permitiendo verificar el grado de cobertura de un módulo en concreto, en función del porcentaje de historias de usuario realizadas.

2.1.5 Validación de Requerimientos

Según Sommerville, el principal objetivo de la validación de requerimientos es el de mostrar que éstos definen el sistema que el cliente desea.

La validación es una actividad muy importante ya que si existen errores en la especificación y se construye el sistema con dichos errores, tras la construcción, el coste de corregirlos será mucho mayor que si se detectan durante la validación de requisitos.

En la siguiente figura se representa el coste relativo de corregir un error: Si el coste relativo de corregir un error en la fase de análisis es 1, el coste de corregirlo en el resto de fases se multiplica por un factor de 3 a 6 en el Diseño, un factor de 10 en la Construcción, un factor de 14 a 40 en las Pruebas de Desarrollo, un factor de 30 a 70 en las pruebas del sistema y entre 40 y 1000 cuando el sistema está en explotación. Por tanto, lo ideal es detectar los errores en tiempo de validación de requisitos ya que de esta forma su corrección sería mucho más eficiente.

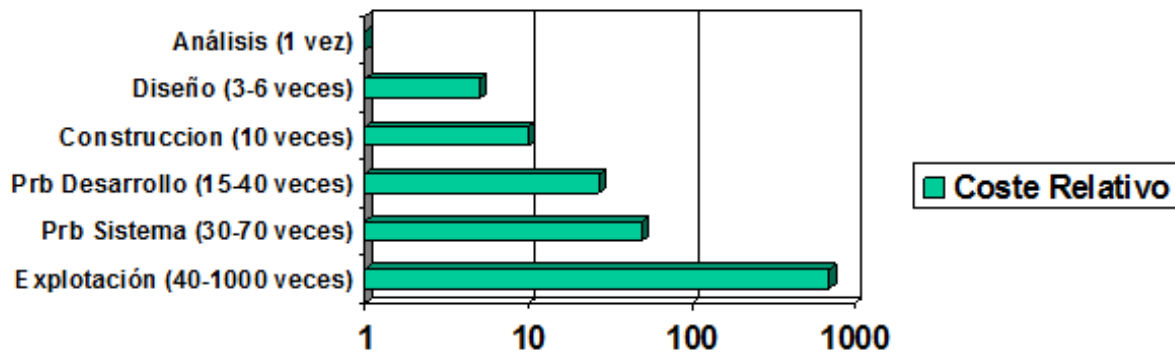


Figura 8 Coste de corrección de errores.

Las verificaciones que se llevan a cabo durante la validación son:

- **Verificaciones de Validez.** Se debe comprobar que los requerimientos de los usuarios describen lo que realmente el usuario necesita. Muchas veces el usuario piensa que necesita una determinada funcionalidad pero realmente, tras un concienzudo análisis, se identifica que necesita funcionalidades adicionales o incluso diferentes. También se deben tener en cuenta que en un mismo sistema suele haber diferentes usuarios, con diferentes puntos de vista, algunas veces contrapuestos, y que éstos deben llegar a un compromiso a la hora de definir los requisitos del sistema.
- **Verificaciones de Consistencia.** Los requerimientos no deben contradecirse.
- **Verificaciones de Completitud.** Los requerimientos deben definir todas las funciones y restricciones propuestas por el usuario del sistema.
- **Verificaciones de Realismo.** Se debe comprobar que los requerimientos se pueden implementar con la tecnología existente en la actualidad.
- **Verificabilidad.** Los requisitos siempre deben ser redactados de forma que se puedan posteriormente verificar, es decir, que se puedan construir un conjunto de pruebas de forma que se pueda determinar si se cumple o no un cierto requisito. Este aspecto toma un valor clave cuando el contrato entre el cliente y el desarrollador se basa en los requerimientos del sistema.

Existen varias técnicas para realizar la validación de los requerimientos. Algunas de ellas se realizan de forma individual mientras que otras se hacen en grupo:

- **Revisión de los Requisitos:** Los requisitos son analizados sistemáticamente por un equipo de revisores.
 - El grupo de revisores estará formado por personal del cliente y de los desarrolladores.
 - Se buscarán omisiones y errores en los requerimientos.
 - Se hace una revisión formal de los requerimientos, explicándose al cliente los requerimientos y las implicaciones de cada uno.
 - Los revisores deben:
 - Comprobar la consistencia de cada requerimiento.

- Comprobar la completitud del conjunto de requerimientos.
- Comprobar la verificabilidad. ¿se puede probar el requerimiento de forma realista?
- Comprobar la comprensibilidad. ¿Comprende el usuario correctamente el requerimiento?
- Rastreabilidad. ¿está claramente establecido el origen del requerimiento? Ante un cambio puede que haga falta ir a la fuente del requerimiento para determinar si se debe cambiar o incluso anular.
- Comprobar la Adaptabilidad. ¿Es adaptable el requerimiento?, es decir, ¿se puede éste cambiar sin que afecte a otros requerimientos del sistema?
- Durante la revisión formal se deben registrar formalmente, en un informe de revisión, los conflictos, contradicciones, errores y omisiones que se identifiquen.
- **Construcción de Prototipos:** Se construye un prototipo del sistema en base a los requerimientos obtenidos. Se le entrega al cliente para que pueda experimentar con él y determinar si es lo que necesita.
- **Generación de Casos de Prueba:** Los requerimientos deben poder probarse (ser verificables). Si se intentan construir casos de prueba para probar los requisitos y los requisitos no son correctos, probablemente se identifiquen problemas a la hora de construir los casos.

Es muy difícil hacer una validación de los requisitos del sistema. Para informáticos expertos en el desarrollo es difícil hacer el modelo mental de todo el sistema en funcionamiento, pero hay que tener muy en cuenta que para los usuarios sea probablemente mucho más difícil. Por estas razones se debe asumir que determinados errores se detectarán en fases posteriores del desarrollo, aunque por lo menos, con estas comprobaciones se conseguirá detectar y corregir el grueso de errores que puedan surgir.

2.1.6 Procedimientos de Gestión de Requisitos

En cada organización se deberían definir un conjunto de procedimientos que permitan especificar de una forma clara las diferentes actividades a realizar en lo referente a la gestión de los requisitos. A continuación se describen los principales procedimientos que se deberían definir y las actividades a realizar. Dependiendo de la magnitud del proyecto, de la magnitud de la organización, etc. estos procedimientos serán más o menos complejos.

- Control de Cambios
- Control de Versiones
- Trazabilidad de Requisitos
- Etc.

La ejecución de los procedimientos genera documentos que contendrán la descripción del cambio, motivo, etc. Esta documentación se deberá guardar, incluso cuando se rechace el cambio, para poder volver a ella más adelante, quizás cuando sea un momento más idóneo para incorporar el cambio, o simplemente para revisar porqué se rechazó un determinado cambio.

2.1.6.1 Gestión de Cambios en los Requisitos

Los cambios en los requisitos son inevitables. Existen cambios en todos los aspectos del sistema durante el desarrollo de éste. Hay que pensar que un sistema grande requiere mucho tiempo de desarrollo, durante ese tiempo se pueden producir múltiples cambios en las necesidades.

Los cambios pueden ser debidos a varias circunstancias, entre las que podemos citar:

- Cambios en el negocio
- Cambios en la reglamentación / legislación
- Nuevas necesidades surgidas de un mejor conocimiento del sistema.

Es imposible congelar los requisitos y continuar con el desarrollo. En ciertos modelos de ciclo de vida, como en el modelo en cascada se hace, pero en la actualidad no es práctico.

Por lo tanto, es necesario controlar los cambios. Para hacerlo hace falta primero analizar el cambio, lo que comporta, la necesidad, el impacto sobre el resto del sistema, etc. y determinar si es realmente interesante hacer el cambio (aceptarlo o rechazarlo). Este análisis lo llevará a cabo un equipo creado específicamente para que realice la gestión de dicho cambio de requisitos. Los cambios pueden implicar grandes modificaciones en los requisitos, por lo que si no se controlan, aunque sean necesarios, pueden dar lugar a constantes modificaciones del sistema, sin lograr llegar a su finalización. Los cambios que se hagan deben ser lo suficientemente justificados como para destinar esfuerzo adicional a la realización de los mismos, posponiéndose el resto para futuras versiones del sistema, cuando éste ya esté estable y el equipo de mantenimiento pueda acometerlos.

Por supuesto, para la gestión de los cambios de los requisitos se hace indispensable un procedimiento por el cual los solicitantes de cada cambio puedan saber qué pasos hay que seguir para notificar el cambio, y que los que controlan los cambios deban saber qué pasos dar si se aprueba o se pospone el cambio.

El control de cambios es un procedimiento que está formado por los siguientes pasos:

- **Propuesta de Cambios.** El afectado por un requisito que no le satisface debe rellenar un formulario de propuesta de cambio indicando cuál es el cambio que hay que realizar. Este cambio se remitirá al equipo de gestión de requisitos para que lo estudie.
- **Análisis de Impactos.** La propuesta de cambio se evalúa para determinar el impacto del cambio en el resto de los requisitos. Hay propuestas cuyo impacto es mínimo, y otras cuyo impacto hace que se deba modificar una parte sustancial del sistema.

Además del impacto se deberá estudiar la oportunidad o necesidad del cambio. Ocurre muchas veces que a un usuario no le parece bien un determinado aspecto del sistema pero a otros sí les satisface. En estos casos se debe llegar a un consenso entre todas las partes.

Otras veces el cambio es interesante pero se considera que no es oportuno hacerlo en dicho momento, quizás porque se quiera lanzar el sistema cuanto antes o porque el cambio es considerable y sería mejor atacarlo en una fase posterior de desarrollo.

- **Toma de Decisiones.** Dependiendo del impacto, de la necesidad u oportunidad del cambio y de otras cuestiones específicas que puedan surgir alrededor del cambio, se debe determinar si el cambio debe ser realizado o no. Si se decide realizarlo se deben hacer las modificaciones pertinentes en el sistema para integrar el cambio con el resto, empezando por los requisitos y finalizando por las partes más avanzadas del sistema.

Si por el contrario se decide no hacerlo se puede optar por dos soluciones:

- Rechazar el cambio, porque se considera que no tiene sentido.
- Posponer su realización para un futuro, se considera el cambio necesario pero el momento no es oportuno. Se pospone para cuando se pueda acometer.
- **Comunicación.** Tanto si se acepta como si no, se debe notificar los efectos de la propuesta de cambio a todos los afectados.
- **Incorporación.** Se hacen las modificaciones pertinentes que se identificaron en el análisis de impacto en los diferentes elementos afectados por el cambio.
- **Medición de la Estabilidad de los Requisitos.** Se evalúan los parámetros que definen la estabilidad de los requisitos tras las modificaciones que se hayan realizado.

2.1.6.2 Control de Versiones

El control de versiones se encarga de mantener en todo momento una versión de los requisitos que sea correcta, coherente y completa. Cuando los requisitos se aprueban, éstos han pasado todas las validaciones y forman un conjunto completo, coherente y correcto. Una vez que se van haciendo cambios en los mismos, si la realización de estos cambios no se controla, y no se sabe si una determinada versión está completa o está siendo cambiada, es muy probable que los desarrolladores se confundan y se basen en requisitos obsoletos.

Por esta razón es necesario definir lo que se denomina una línea base. Esta línea base constituye el marco de referencia estable para el desarrollo del producto, conteniendo un conjunto de requisitos completo, coherente y correcto.

IEEE define el concepto de línea base como una especificación o producto que ha sido revisado formalmente y sobre la que se ha llegado a un acuerdo, y que de ahí en adelante sirve como base para un desarrollo posterior y que puede cambiarse solamente a través de procedimientos formales de control de cambios.

Los desarrolladores se basarán en la línea base, que tiene las últimas versiones aprobadas de los requisitos. El hecho de que la línea base sea una versión congelada de los requisitos no implica que estos no puedan cambiarse. Si un requisito se quiere cambiar, se realizarán las modificaciones pertinentes y cuando ya estén finalizadas y aprobadas se procederá a cambiarlo en la línea base. Acto seguido se notificará la existencia de una nueva línea base para que los desarrolladores lo tengan en cuenta.

Las tareas más importantes en el control de versiones son las de controlar los documentos que se están usando para el desarrollo. Para ello:

- Se empleará una herramienta automática de control de versiones para controlar todas y cada una de las versiones de los documentos que se generen durante el desarrollo y mantenimiento del sistema. Se pueden usar para ello herramientas como CVS, Subversion, PVCS, MS Source Safe, SCCS, etc. Algunas herramientas son de software libre, como CVS o Subversion, mientras que otras como PVCS o MS Source Safe son de pago.

Cualquiera de estas herramientas permite llevar un control de la línea base, poder ver el histórico de modificaciones de cada elemento controlado por el sistema y obtener la última versión de un elemento. Cuando varias personas quieren trabajar sobre un mismo elemento, éstas obtienen el de la línea base, realizan sus modificaciones y según van terminando van integrando sus cambios en la versión de la línea base. Esta versión de la línea base podrá ser la que ellos obtuvieron originalmente al empezar el cambio, o una diferente que puede haber cambiado porque alguna otra persona ya haya incorporado antes sus cambios.

- Se debería poder acceder siempre a la última versión y a versiones anteriores de cada documento. De esta forma, si se detecta un fallo en una versión de un documento es posible volver atrás y recuperar los datos originales.
- Todos los cambios que se hagan deben documentarse y comunicarse a los afectados, para que todo el mundo trabaje con la última versión. Es especialmente importante el indicar en cada nueva versión qué cambios sufre respecto a la anterior, para evitar que tenga que revisarse el documento entero.
- Cada documento debería tener:
 - Historia de revisiones que identifique los cambios realizados
 - Fecha del cambio, responsable y motivos.

2.1.6.3 Aproximación a los cambios de requerimientos en Metodologías Ágiles

Aunque se tratarán más adelante en este tema las Metodologías Ágiles se considera adecuado hacer un breve inciso en este capítulo. Una de las principales diferencias de las metodologías ágiles y las metodologías tradicionales es la manera en que tratan los requisitos y, sobre todo, cómo responden a los cambios.

Para las metodologías ágiles, el desarrollo de software debe proporcionar valor, considerando que para ello se debe empezar por aquellos requisitos que tienen la máxima prioridad: aquellos que maximizan el ROI (Return of Investment o retorno de la inversión).

Con respecto a los cambios en los requisitos, las metodologías ágiles asumen el cambio como inevitable, por tanto no intentan evitarlos sino que intentan gestionar el cambio. Todas las metodologías ágiles tienen en común la aceptación de los cambios en los requisitos como algo inherente al desarrollo software, aunque cada metodología tiene sus propios métodos de respuesta a

estos cambios, como ejemplo del manejo de requisitos se describirá la estrategia que se sigue en Scrum.

2.1.6.3.1 Gestión de Requisitos en Scrum

Los requisitos forman parte del Product Backlog, donde se encuentran priorizados:

- El Product Backlog es la lista de requisitos, debe estar ordenada por prioridades de tal forma que los más prioritarios estarán más detallados y los menos prioritarios lo estarán menos.
- Los requisitos pueden aparecer en cualquier momento, cuando aparecen se priorizan y se introducen en el lugar adecuado del Product backlog. Los stakeholders pueden solicitar nuevos requerimientos, pero la autoridad final para la priorización será el Product Owner (quien representa a todos los stakeholders).
- Además, los requisitos pueden ser repriorizados en cualquier momento (esto quiere decir que cambiarán de posición en el Product Backlog).
- En cada Sprint se elegirá un conjunto de requisitos de la parte alta de la lista y se implementarán: en el caso de Scrum estos requisitos no pueden ser modificados durante el Sprint.

2.1.6.4 Trazabilidad de Requisitos

El concepto de trazabilidad hace referencia a la posibilidad de determinar cómo se ha llegado a un cierto elemento del software a partir de otros. Para poder llevar a cabo esta trazabilidad es especialmente importante el poder relacionar unos requisitos con otros y también relacionar requisitos con elementos del sistema a los que da lugar.

Se pueden dar dos tipos de trazabilidad:

- **Hacia delante:** Partiendo de un requisito se llega a todos los elementos que materializan dicho requisito, ya sea del análisis, del diseño, de la construcción, de pruebas, etc-
- **Hacia atrás:** Partiendo de un elemento del sistema se llega al requisito que lo generó.

Esta relación también permite poder hacer validaciones para comprobar que todos los requisitos del sistema dan lugar a al menos una parte del sistema, y que todas las partes del sistema están basadas al menos en un requisito. Si esto no se cumple, es porque o el requisito sobra, ya que no da lugar a ninguna parte del sistema, o en el otro caso, la parte del sistema sobra porque no está soportada por ningún requisito.

Además, un requisito se debe poder relacionar con otros requisitos similares para así evitar repeticiones y facilitar su actualización. Además, así cuando se hace un cambio en uno, es más fácil localizar aquellos requisitos en los que hay relación, para ver el impacto del cambio.

La trazabilidad suele representarse con la llamada matriz de trazabilidad, según PMI (Project Management Institute), la matriz de trazabilidad es una tabla que vinculará cada requisito con los entregables del proyecto donde éste se refleja. Se trata de un documento muy útil ya que permite, de manera rápida, tener una visión global de los requisitos y ayuda a identificar el impacto de los cambios en los requisitos.

2.1.7 El Análisis de Requisitos en Métrica versión 3

Aunque Métrica v3 es objeto de otro tema de la convocatoria, se estima oportuno hacer una breve mención a las labores de análisis de requisitos incluidas en esta metodología.

Métrica v3 dedica una actividad al análisis de requisitos, dedicando cuatro actividades adicionales para la modelización del sistema. Dos de estas actividades se realizan cuando se usa una aproximación estructurada, las actividades de confección del modelo de datos y modelo de procesos, mientras que las otras dos se usan con una aproximación orientada a objetos, confeccionándose en

este caso los casos de uso y el diagrama de clases. En esta explicación nos centraremos en la parte correspondiente a la obtención y análisis de requisitos.

En la actividad “ASI 1. Definición del Sistema” se procede a definir el sistema, los límites de éste y los interfaces con otros sistemas. Como parte de esta actividad se confecciona el catálogo de requisitos del sistema, para lo cual se partirá del catálogo de requisitos que se identificó en la fase EVS de Estudio de la Viabilidad del Sistema. Este catálogo de requisitos será el germen a partir del cual se generará en la actividad siguiente el catálogo de requisitos del software.

El Análisis de Requisitos propiamente dicho se lleva a cabo en la actividad “ASI 2. Establecimiento de Requisitos”. A lo largo de esta actividad se lleva a cabo la definición, análisis y validación de los requisitos a partir de la información facilitada por los usuarios. El objetivo principal de esta actividad es el de confeccionar un catálogo detallado de requisitos.

Como técnica para ayudar a la obtención de requisitos se propone el uso de la técnica de Especificación de los Casos de Uso, que es obligatoria para la aproximación orientada a objetos, pero que es opcional para el enfoque estructurado.

La actividad “ASI 2. Establecimiento de Requisitos” se subdivide en cuatro tareas, que no requieren ser ejecutadas estrictamente en un orden secuencial, sino que más bien se ejecutan de forma iterativa exigiendo continuas realimentaciones y solapamientos.

Las cuatro tareas que se definen dentro de la actividad ASI 2 son las siguientes:

- **ASI 2.1: Obtención de Requisitos.** En esta tarea se realiza la obtención de los requisitos mediante reuniones con los usuarios. Se definen las prioridades a los requisitos que se van obteniendo y se identifican los diferentes casos de uso.
- **ASI 2.2: Especificación de Casos de Uso.** Se especifica cada caso de uso que se haya identificado. Es opcional para el enfoque estructurado.
- **ASI 2.3: Análisis de Requisitos.** Se estudia la información capturada en las tareas anteriores en busca de inconsistencias, duplicidades, ambigüedades, falta de información, etc. Además se analizan las prioridades definidas por el usuario y se relacionan unos requisitos con otros. Mediante sesiones de trabajo, se contrastan las conclusiones con los usuarios.
- **ASI 2.4: Validación de Requisitos.** Se comprueba junto con los usuarios que el catálogo de requisitos generado y los casos de uso son válidos, consistentes y completos.

En la guía de técnicas y prácticas de Métrica v3 se incorporan un conjunto de técnicas para poder obtener unos buenos resultados a la hora de realizar un análisis de requisitos. Técnicas como las entrevistas, factores críticos de éxito, casos de uso, ya vistas anteriormente en este tema, son algunas de las técnicas descritas por Métrica v3 y usadas en esta actividad de Análisis de Requisitos.

Como resumen se muestran en la siguiente tabla las tareas, productos, técnicas, prácticas y participantes de la actividad.

Tarea		Productos	Técnicas y Prácticas	Participantes
ASI 2.1	Obtención de Requisitos	• Catálogo de Requisitos • Modelo de Casos de Uso	• Sesiones de Trabajo • Catalogación • Casos de Uso	• Usuarios Expertos • Analistas
ASI 2.2	Especificación de Casos de Uso	• Catálogo de Requisitos • Modelo de Casos de Uso • Especificación de Casos de Uso	• Sesiones de Trabajo • Catalogación • Casos de Uso	• Usuarios Expertos • Analistas
ASI 2.3	Análisis de Requisitos	• Catálogo de Requisitos • Modelo de Casos de Uso • Especificación de Casos de Uso	• Sesiones de Trabajo • Catalogación • Casos de Uso	• Usuarios Expertos • Analistas
ASI 2.4	Validación de Requisitos	• Catálogo de Requisitos • Modelo de Casos de Uso • Especificación de Casos de Uso	• Sesiones de Trabajo • Catalogación • Casos de Uso	• Usuarios Expertos • Analistas

3 Metodologías de desarrollo de sistemas

Las metodologías sistematizan el proceso de desarrollo de las aplicaciones para que los diferentes participantes en el desarrollo sean capaces de tener la misma visión. Las metodologías son un conjunto de pasos, que partiendo de cero permiten construir un sistema mediante la ejecución ordenada de dichos pasos. La metodología indica, para cada uno de estos pasos, las actividades que se deben ir haciendo y qué técnicas se deben aplicar.

Muchas veces, especialmente los desarrolladores que están acostumbrados al desarrollo sin ningún tipo de metodología o estándar, tienden a pensar que el uso de metodologías es una sobrecarga inútil en el esfuerzo que hay que realizar para construir el sistema. Es cierto que una metodología impone la construcción de una serie de documentos, modelos, etc. y que estos requieren tiempo para confeccionarse, pero también es cierto que esta documentación ayuda a irse haciendo con el problema, y si está correctamente confeccionada y actualizada con los cambios que van surgiendo, es muy útil para las tareas de mantenimiento. Además, la documentación se debería realizar en algún momento, con lo que este esfuerzo no debería ser adicional en la mayor parte de los casos.

Otra de las principales críticas que se suelen hacer a las metodologías es que para proyectos pequeños hay que realizar excesivas tareas y se requiere la participación de mucho personal. La metodología debe ser vista siempre como una ayuda, y no como una traba al desarrollo. Muchas de las metodologías son flexibles, como ocurre por ejemplo con Métrica v3, que permite adaptar las actividades que hay que realizar a un proyecto en particular, es decir, se pueden quitar o modificar actividades según el proyecto. De esta forma, un pequeño proyecto que requiere una persona durante dos meses no se incrementaría excesivamente por el uso de una metodología ni requeriría la confección de innumerables documentos.

3.1 Definición de Metodología

Existen múltiples definiciones del concepto de metodología, todas son parecidas, y en el fondo hacen referencia a los mismos conceptos. Una posible definición es:

Metodología: Conjunto de procedimientos, métodos o técnicas y herramientas y un soporte documental que ayuda a los desarrolladores a realizar nuevo software.

El objetivo de una metodología es el de cubrir las siguientes necesidades:

- Mejorar el desarrollo de las aplicaciones. Permitir desarrollar aplicaciones que tengan una mejor calidad, que se desarrollen con los costes y plazos presupuestados, que cumplan con las necesidades de los usuarios.
- Un proceso de desarrollo que identifique productos intermedios en cada fase de forma que se pueda planificar y controlar el proceso.
- Un proceso estándar en la organización. Todos los componentes de la organización deberían usar dicho proceso, que al ser estándar les será mucho más fácil incorporarse a un nuevo proyecto que si cada proyecto sigue su metodología, o incluso peor, no se sigue ninguna.

A grandes rasgos, una metodología define una serie de pasos para construir el software. Los componentes principales son:

- Un **conjunto de procedimientos**, que vienen a ser un conjunto de pasos a realizar, según un orden preestablecido, para completar el proyecto.

Así por ejemplo, una metodología tiene una fase de Análisis, otra de Diseño, otra de Construcción, etc. Dentro de cada fase se realizan una serie de pasos, por ejemplo en el análisis: obtener los requisitos del sistema, confeccionar el modelo de datos o confeccionar el modelo de procesos.

Estos pasos están definidos con una determinada secuencia y no pueden realizarse unos hasta que los anteriores no están completados, principalmente porque los pasos que vienen detrás se basan en la información generada en los pasos anteriores.

- Los **métodos o técnicas** que indican cómo se debe construir el software. Para cada uno de

los procedimientos anteriores existe un conjunto de técnicas que se pueden aplicar para completar cada paso.

Así por ejemplo, para la confección del modelo de datos se puede usar la técnica del Modelo Entidad / Relación o la técnica del Diagrama de Clases.

- Las **herramientas** permiten la automatización de los métodos o técnicas usando programas de ordenador específicos.

Continuando con el ejemplo, se pueden usar herramientas informáticas como CA ERWin Data Modeler Platinum, Microsoft Visio, Oracle Designer, etc. para hacer la construcción del modelo de datos, u otros diagramas según las funcionalidades de las herramientas.

Después de esta visión general de cada uno de los componentes de una metodología, en los siguientes apartados se va a entrar en más detalle en cada uno de los tres aspectos anteriores.

3.1.1 Procedimientos

Los procedimientos son los pasos que hay que dar para construir el software. Estos procedimientos son los elementos que unen a las técnicas y las herramientas.

Los procedimientos definen:

- La secuencia en que se aplican las técnicas: define qué actividades se deben realizar y en qué orden. El orden no es arbitrario sino que depende de las necesidades de información de cada actividad a realizar. Así por ejemplo, si hay que construir el modelo físico de datos es impensable poderlo hacer si antes no hemos construido el modelo lógico de datos, pues este es la base de partida.
- Los productos que se deben obtener (documentos, informes, etc.). El procedimiento especifica qué documentos se deben generar y cuáles serán los documentos o la información de partida a partir de la que se generarán.
- Los controles: Ayudan a asegurar la calidad y coordinar los cambios.
- Las Directrices: Ayudan a los gestores del software a evaluar el progreso.

3.1.2 Métodos o Técnicas

Los métodos o técnicas indican cómo construir técnicamente el software de manera que se obtenga un software de calidad a un coste aceptable. Cada técnica está enfocada para resolver un problema determinado y deberán ser los responsables del proyecto los que determinen qué conjuntos de técnicas deben aplicarse en cada momento. Así por ejemplo, si el proyecto se plantea desarrollarse con una aproximación estructurada se usarán las técnicas estructuradas. Si por el contrario se plantea usar una aproximación orientada a objetos, se deberán usar las técnicas orientadas a objetos.

Un Método o Técnica incluye los siguientes elementos:

- **Descripciones del Modelo del Sistema:** Descripciones de los modelos del sistema que se desarrollará y la notación utilizada para definir estos modelos. Por ejemplo, modelos de objetos, de flujo de datos, de máquina de estados, etc.
- **Reglas:** Restricciones que siempre aplican a los modelos de sistemas. Por ejemplo, cada entidad de un modelo de sistema debe tener un nombre único.
- **Guías de Proceso:** Descripciones de las actividades que deben seguirse para desarrollar los modelos del sistema y la organización de estas actividades. Por ejemplo, imponer que los atributos de los objetos deban ser documentados antes de definir las operaciones asociadas al objeto.
- **Recomendaciones:** Se trata de consejos, basados en la experiencia, sobre cómo aplicar ese método o técnica.

Son ejemplos de métodos o técnicas:

- Técnicas PERT / CPM / WBS para la Planificación y Estimación de Proyectos.

- Técnica del Modelo Entidad Relación para la construcción del Modelo de Datos.
- Técnica de los DFDs para la construcción del modelo de procesos (DFD).
- Técnica de Diagramas de Estructura para definir la arquitectura del sistema.

Hay metodologías como Métrica v3 que permiten usar diferentes aproximaciones para el desarrollo del proyecto. Así por ejemplo, se permite realizar una aproximación estructurada, con lo que se usarán los métodos o técnicas estructuradas, o también se puede optar por una aproximación orientada a objetos, con lo que se usarán los métodos o técnicas orientadas a objetos.

3.1.3 Herramientas CASE

Se puede definir CASE (Computer-Aided Software Engineering) como la aplicación de la tecnología informática a las actividades, las técnicas y las metodologías propias del desarrollo de sistemas. Las herramientas CASE son aplicaciones informáticas que automatizan o apoyan una o más fases del ciclo de vida del desarrollo del software

Las herramientas CASE se pueden clasificar, según la fase dentro del ciclo de vida de desarrollo del software en:

- **Upper CASE o CASE Superior:** Son de muy alto nivel y dan soporte a la Planificación Estratégica y Gestión del Proyecto.
- **Middle CASE o CASE Intermedio:** Referidas a la Automatización del Análisis y el Diseño del Sistema.
- **Lower CASE o CASE Inferior:** Referidas a la Automatización de las fases finales del desarrollo como la programación, la generación de código, generación de pruebas o mantenimiento.
- **Herramientas de Ingeniería Inversa:** Ayudan al analista, en las situaciones de falta de documentación o desactualización de ésta, a interpretar el código fuente, reconstrucción del modelo de clases, definiciones de esquemas de bases de datos, etc.
- **Herramientas de Ingeniería de Ida y Vuelta:** Son un híbrido entre las Lower CASE y las de Reingeniería. La característica más importante que describe a estas herramientas es que soportan su representación interna en el código fuente, con lo que el diseño y el código fuente siempre están en concordancia. La herramienta genera e interpreta código en tiempo real.
- **Herramientas de Ámbito Específico.** Son herramientas que dan soporte a un ámbito muy específico.

Las herramientas CASE también pueden clasificarse según dan un soporte parcial o completo al desarrollo:

- **Tool CASE:** Da soporte a una parte del ciclo de desarrollo. Estas herramientas no se integran con otras.
- **I-CASE:** Da soporte a todo el proceso de desarrollo de software. Las herramientas están unas integradas con otras y las salidas de unas alimentan a las siguientes. Entre estas herramientas se encuentran Rational Rose (IBM) o Together Control Center (Borland).

3.1.4 Soporte Documental

Define un conjunto de documentos o entregables que se deben ir confeccionando durante el proceso de desarrollo. Las salidas de un paso anterior se incorporan en el siguiente. Además, toda la documentación que se genera sirve para que otras personas que entren en el proyecto puedan ponerse al día sobre el mismo o durante el mantenimiento, para que el equipo de mantenimiento pueda acceder y conocer los detalles del sistema, aunque no haya participado en el desarrollo del mismo.

Como se ha visto anteriormente, si volvemos a la especificación de la tarea DSI 5.1, Métrica define una serie de documentos a generar en cada paso que se da. En definición de las tareas se especifican los productos que hay que generar (apartado Productos de Salida) y a partir de cuáles se debe generar. En el apartado de Productos de Entrada no sólo se especifican los productos a utilizar en la tarea, sino que como ayuda, entre paréntesis también se especifica la tarea en la que se han generado.

3.1.5 Requisitos de una Metodología

Según Dorfman, una metodología, para poderse llamar como tal, debe cumplir los siguientes requisitos:

- **Documentada:** El procedimiento de uso de la metodología debe estar descrito en un documento o manual de usuario.
- **Repetible:** Se debe poder aplicar varias veces la metodología.
- **Enseñable:** El detalle con el que se ha definido la metodología es suficiente y existen ejemplos para que se pueda enseñar la metodología.
- **Basada en Técnicas Probadas:** Una metodología debe basarse en técnicas ya existentes, de las que se conoce su utilidad o en otras metodologías más simples.
- **Validada:** La metodología debe haberse usado satisfactoriamente en un gran número de aplicaciones.
- **Apropiada:** La metodología es apropiada para el problema que se quiere resolver.

3.1.6 Características de las Metodologías

Una metodología que cumple los requisitos anteriores tiene una serie de características, algunas de éstas se definen a continuación:

- **Existencia de reglas predefinidas** que permiten determinar en cada momento qué es lo que se debe hacer, siguiéndose un orden en la realización de las actividades de forma que se garantice una coherencia entre los resultados obtenidos en un paso anterior y los obtenidos en otro posterior. Estas reglas predefinidas deben estar documentadas en un documento o manual de usuario.
- **Cobertura total del ciclo de desarrollo del sistema.** La metodología cubre todo el ciclo de vida del sistema, desde su concepción hasta su retirada del servicio. Por lo tanto, una metodología no sólo debe cubrir el desarrollo del software, sino que también el mantenimiento posterior del mismo.

Algunas de las metodologías más antiguas sólo cubrían la parte de desarrollo del software, pero no contaban con aspectos como el plan de sistemas de información o el mantenimiento por poner unos ejemplos.

- **Verificaciones intermedias.** Al final de cada fase se hacen unas validaciones y verificaciones intermedias para comprobar que los productos obtenidos son correctos y son los que deben ser. Con estas validaciones y verificaciones se pretende asegurar que el producto es el correcto, es decir, es el producto que el usuario necesita y que se está construyendo correctamente. Para ello se comprueba que se están siguiendo los pasos de la metodología y que los productos, tanto intermedios como finales, obtenidos durante el desarrollo guardan una coherencia, derivándose unos de otros.
- **Planificación y Control.** Una metodología permite planificar y controlar la ejecución del proyecto. Las fases intermedias, entregas de documentación, validaciones y verificaciones permiten poder determinar diferentes hitos en la planificación de forma que sea posible determinar el grado de avance del proyecto, y por tanto si existen desviaciones respecto al plan previsto.
- **Comunicación efectiva.** La metodología permite una comunicación efectiva entre todos los participantes, es decir, entre desarrolladores y usuarios, para lograr un común entendimiento

y que la visión de los usuarios sea entendida y plasmada por los desarrolladores en la aplicación que están construyendo.

- **Utilización sobre un abanico amplio de proyectos.** Una metodología debe poderse usar sobre un conjunto amplio de proyectos, es decir, debe ser un método general de construir aplicaciones, no un método para construir una aplicación en particular.
- **Fácil formación.** La metodología debe ser fácil de enseñar, para que los desarrolladores puedan entenderla y aplicarla a su trabajo.
- **Documentación.** Las metodologías permiten ir documentando el sistema a medida que se va desarrollando, con lo que al llegar al final del sistema se dispone de una documentación completa de todo el sistema, la cual será de gran utilidad para la fase de mantenimiento. Por supuesto, durante el mantenimiento se deberá modificar correspondientemente la documentación de desarrollo para adaptarla a las modificaciones que sufra el software durante dicha fase.
- **Herramientas CASE.** Si es posible, la metodología debería estar soportada por alguna herramienta CASE del mercado, y además sería deseable que se dispusiese de alguna herramienta para aplicarla. Esto no implica que tenga que existir una herramienta específica en el mercado para una determinada metodología, sino que debería existir alguna herramienta que al menos permita configurarse para poder seguir la metodología.
- **Debe contener actividades que mejoren el proceso de desarrollo.** Las actividades que se lleven a cabo en la metodología deben mejorar el proceso de desarrollo, para aumentar la calidad de los desarrollos, para aumentar la productividad y simplificar la coordinación entre los diferentes participantes.
- **Soporte de la reutilización de software.** Una metodología debe permitir reutilizar el software que se desarrolla. De esta manera se logra reducir el tiempo de desarrollo y la cantidad de errores cometidos al reutilizar elementos ya construidos, utilizados y probados previamente.

3.1.7 Metodología en comparación con Ciclo de Vida

Antes de continuar estudiando las metodologías hay que ver las similitudes y diferencias entre una Metodología y un Ciclo de Vida. ¿Son lo mismo? ¿En qué se diferencian?

Se puede definir cada uno de ellos como:

- **Metodología:** Define los pasos, detalla los productos, técnicas y métodos que hay que seguir para construir el sistema software.
- **Ciclo de Vida o Modelo de Proceso:** Define las etapas y actividades genéricas por las que pasa el desarrollo de un sistema software (ciclo de vida en cascada, en espiral, etc.).

En ambos casos se define un conjunto de pasos o actividades por las que pasa el proyecto. La diferencia principal está en que la metodología especifica en detalle qué pasos hay que dar y cómo deben ser éstos dados. Es decir, explicita por ejemplo que hay que construir la arquitectura del sistema software y para ello se usará la técnica de Diagrama de Cuadros, se generará un Diagrama de Cuadros y para ello se partirá de una serie de documentos preparados previamente. Como se ve, especifica claramente lo que hay que hacer.

En cambio, el ciclo de vida especifica las fases por las que pasa un proyecto, es decir, cómo evoluciona el proyecto a lo largo del tiempo. Se indica que hay una fase de análisis, una de diseño, etc, pero a diferencia de las metodologías, no se especifica qué se debe hacer o qué técnica aplicar en cada momento para continuar con el desarrollo.

Las metodologías y los ciclos de vida están relacionados pues una metodología puede seguir uno o más ciclos de vida. Al igual, un ciclo de vida no es exclusivo de una metodología, por lo que varias metodologías pueden seguir el mismo ciclo de vida.

3.2 Clasificación de las Metodologías

Hoy en día existen múltiples metodologías, unas son de tipo general, otras sin embargo son más específicas para un determinado ámbito, como por ejemplo la programación de sistemas expertos o el desarrollo de software de control para autómatas programables en instalaciones industriales. En una de las múltiples clasificaciones que se pueden hacer, en la línea de lo comentado a lo largo del tema, las diferentes metodologías existentes se pueden clasificar según los siguientes tipos:

- Metodologías Estructuradas
 - Orientadas a Procesos
 - Orientadas a Datos
 - Orientadas a Datos Jerárquicos
 - Orientadas a Datos no Jerárquicos
- Metodologías Orientadas a Objetos
- Para Sistemas de Tiempo Real
- Metodologías Ágiles

A continuación se va a estudiar cada una de ellas con un poco más de profundidad.

3.2.1 Metodologías Estructuradas

El enfoque estructurado considera los datos y el proceso que transforma los datos como entidades separadas. Los objetos de datos se modelan en una forma que define sus atributos y relaciones. Los procesos que manipulan los objetos de los datos se modelan de tal manera que muestran cómo se transforman los datos, mientras fluyen estos por el sistema.

El análisis estructurado permite al analista conocer un sistema o proceso (actividad) en una forma lógica y manejable al mismo tiempo que proporciona la base para asegurar que no se omite ningún detalle pertinente. El objetivo principal es el de organizar las tareas relacionadas con la determinación de los requisitos para lograr una comprensión completa y exacta de la situación.

Las metodologías estructuradas proponen la creación de modelos del sistema que representan:

- Los Procesos que se realizan en el sistema, es decir, qué transformaciones se realizan.
- Los Flujos de datos, es decir, cómo fluyen los datos entre los procesos del sistema.
- La Estructura de los Datos: qué atributos componen los datos y qué relaciones tienen unos datos con otros.

Todas estas metodologías se apoyan en un enfoque Top-Down mediante el cual se parte del problema como un todo y mediante la subdivisión de cada problema en subproblemas más pequeños se va bajando en el nivel de abstracción hasta llegar al nivel inferior de detalle.

La diferencia principal entre las metodologías estructuradas es el objeto sobre el que se centran a la hora de construir el sistema, es decir, si se centran en el proceso (metodologías orientadas al proceso) o se centran en los datos (metodologías orientadas a datos).

3.2.1.1 Metodologías Estructuradas Orientadas a Procesos

Estas metodologías se basan en el modelo básico de entrada / proceso / salida, centrándose principalmente en la parte del proceso, es decir, lo que el sistema hace y cómo procesa la entrada para generar la salida.

La especificación del sistema se basa en describirlo en función de:

- Diagramas de Flujo de Datos
- Diccionario de Datos
- Especificaciones de Procesos

Para la confección de los modelos, estas técnicas se apoyan en técnicas gráficas para obtener una especificación estructurada, es decir, un modelo gráfico, particionado, descendente y jerárquico de los procesos del sistema y los datos utilizados por dichos procesos.

Los **Diagramas de Flujos de Datos**, que se explicarán brevemente ya que existe otro tema en el que se revisarán con más detalle, permiten hacer una especificación estructurada del sistema ya que cada elemento del DFD se puede expandir en un nuevo diagrama en donde se detalle dicho elemento. Esto da lugar a una jerarquía de diagramas en el que cada elemento del diagrama se divide y describe en más detalle en el diagrama de nivel inferior. De forma iterativa y descendente se va entrando en detalle en cada uno de los elementos de los diagramas hasta que se llega al nivel más inferior, en el que ya no se puede expandir más el diagrama, es ese nivel se pasa a crear la **Especificación de Procesos**.

El proceso comienza con el DFD de nivel 0 o diagrama de Contexto: Se dibuja un único proceso que representa el sistema y las entidades externas que corresponden con los usuarios o con los sistemas con el que se comunica el sistema que se está diseñando. El intercambio de información se representa mediante las flechas.

El siguiente paso es descomponer este diagrama en distintos niveles de manera que la descomposición jerárquica nos permite entrar en más detalle en un aspecto del sistema.

El proceso finaliza cuando hemos llegado a un proceso que no tiene sentido seguir descomponiendo puesto que ya es lo suficientemente claro. A estos procesos se los denomina Procesos Primitivos y para ellos se escribirá la **Especificación del Proceso**.

En paralelo a la construcción de los DFDs se elabora el **diccionario de datos**. El diccionario de datos es el repositorio común en el que está definido cada dato que fluye por el sistema. Los datos se definen una vez y se usan en todos aquellos lugares del sistema donde son necesarios. De esta forma se evitan repeticiones de los datos y se tienen correctamente documentados.

Algunos autores que han desarrollado las metodologías estructuradas orientadas a procesos son, entre otros, DeMarco, Gane & Sarson o Yourdon. A continuación se muestra como ejemplo los pasos que se deben dar en diferentes metodologías estructuradas:

DeMarco	Gane & Sarson	Yourdon / Constantine
1. Construir el modelo físico actual (DFD físico actual) 2. Construir el modelo lógico actual (DFD lógico actual) 3. Crear un conjunto de modelos físicos alternativos 4. Estimar los costes y tiempos de cada opción 5. Seleccionar un modelo 6. Empaquetar la especificación	1. Construir el modelo lógico actual (DFD lógico actual) 2. Construir el modelo del nuevo sistema: elaborar una especificación estructurada y construir un modelo lógico de datos en tercera forma normal que exprese el contenido de los almacenes de datos. 3. Seleccionar un modelo lógico 4. Crear el nuevo modelo físico del sistema 5. Empaquetar la especificación	1. Realizar los DFD del sistema 2. Realizar el diagrama de estructuras 3. Evaluar el diseño 4. Preparar el diseño para la implantación

Como se puede apreciar, las tres metodologías se basan en el uso de Diagramas de Flujo de Datos o DFDs, originalmente propuestos por DeMarco, para la construcción de los modelos gráficos de representación del sistema.

3.2.1.2 Metodologías Estructuradas Orientadas a Datos

Las técnicas orientadas a datos establecen modelos de sistemas que se basan en una organización y un acceso ideales a los datos del sistema, independientemente de cómo se utilicen estos datos para

satisfacer las necesidades de información (salidas). Las metodologías orientadas a estructura de datos representan los requisitos del software orientándolos hacia la estructura de datos, en vez del flujo de datos.

El objetivo principal de estas metodologías es el de transformar la representación de la estructura de datos en la representación de la estructura del software que debe manejarlos. El primer paso que se debe realizar es la modelización de los datos. Una vez disponible la modelización de datos se genera la estructura del software que debe manejarlos. Para ello se dispone de tres estructuras básicas, que son la secuencia, la selección y la repetición.

Existen dos tipos de metodologías orientadas a datos, en función de si los datos con los que tratan son o no jerárquicos:

- Orientadas a Datos Jerárquicos
- Orientadas a Datos no Jerárquicos

3.2.1.2.1 Metodologías Estructuradas Orientadas a Datos Jerárquicos

Las metodologías orientadas a datos se basan en modelar el sistema en función de la estructura de los datos que el sistema trata. En muchas áreas de aplicación existe una estructura de información bien diferenciada y jerárquica, es decir, los datos de entrada, las estructuras internas de información y los datos de salida se pueden representar en estructuras bien diferenciadas.

A grandes rasgos, la forma de trabajar con estas metodologías es: primero definir las estructuras de datos y a partir de estas se generan los componentes procedimentales que definen la estructura de control del programa.

- La estructura de control del programa debe ser jerárquica y se deriva de la estructura de datos
- El proceso de diseño consiste primero en definir las estructuras de los datos de entrada y salida, mezclarlas todas en una estructura jerárquica de programa y después ordenar detalladamente la lógica procedimental para que se ajuste a la estructura.
- El diseño lógico debe preceder y estar separado del físico

Aunque cada metodología tiene sus características específicas de notación y enfoque se pueden identificar las siguientes características comunes:

- Todas delegan en el analista la identificación de los objetos de información clave (también llamadas entidades) y las operaciones (también llamadas acciones o procesos).
- Todas representan la estructura de la información mediante la secuencia, la selección y la repetición.
- Todas definen una secuencia de pasos para convertir la estructura jerárquica de la información en una estructura de programa.

Estas metodologías se desarrollaron a finales de los años 70 y principios de los 80 por autores como Warnier, Cameron, o Jackson. Son ejemplos de dichas técnicas las siguientes:

- J.D. Warnier desarrolló el CLP (Construcción Lógica de Programas), que derivó en DSED (Desarrollo de Sistemas Estructurados en Datos) o método Warnier-Orr.
- Jackson desarrolló dos metodologías:
 - JSP (Jackson Structured Programming): Método para el diseño de programas mediante la composición de procesos secuenciales.
 - JSD (Jackson System Development): Permite especificar y diseñar sistemas cuyo comportamiento se basa en secuencias de eventos.

3.2.1.2.2 Metodologías Estructuradas Orientadas a Datos no Jerárquicos

En estas metodologías los datos son el corazón del sistema de información. El modelo de datos está formado por un conjunto de entidades básicas y las interrelaciones entre dichas entidades.

Un ejemplo de estas metodologías es la metodología “Ingeniería de la Información”. Esta metodología es claramente orientada a datos ya que da una mayor preponderancia a los datos sobre los procesos, aunque no se olvida de analizarlos.

Se definen 4 fases:

- **Planificación:** construir una arquitectura de la Información y una estrategia que soporte los objetivos de la organización
- **Análisis:** comprender las áreas del negocio y determinar los requisitos del sistema.
- **Diseño:** establecer el comportamiento del sistema deseado por el usuario y que sea alcanzable por la tecnología
- **Construcción:** construir sistemas que cumplan los tres niveles anteriores

Estas metodologías han sido desarrolladas por autores como James Martin o Clive Finkelstein.

3.2.2 Metodologías Orientadas a Objetos

El paradigma de la Orientación a Objetos surge a finales de los años 60, aunque no es hasta mediados de los 80 cuando se empieza a generalizar su uso. El concepto de Orientación a Objetos presenta un cambio sustancial en las metodologías existentes hasta el momento. En el enfoque estructurado se plantea modelar el sistema desde el punto de vista de las funciones y de los procesos. El modelado orientado a objetos, sin embargo, se centra en modelar el problema como un conjunto de objetos que colaboran e interactúan entre ellos para lograr implementar los requisitos del cliente. El concepto de objeto es una abstracción que encapsula las funciones y los datos.

Las metodologías orientadas a objetos se plantean desde dos enfoques bien diferentes:

- Un enfoque revolucionario o puro, que se entiende como un cambio profundo en las metodologías estructuradas, las cuales se ven obsoletas. Ejemplos de estas metodologías son OOAD de Booch o CRC/RDD (Wirfs-Brock).
- Un enfoque evolutivo, en las que el análisis y diseño estructurado se usan como base para el desarrollo de la orientación a objetos. Ejemplos de estas metodologías son OMT y PUDS/RUP (RUP es una adaptación de IBM del PUDS).

Las metodologías orientadas a objetos se caracterizan por:

- El dominio del problema es un conjunto de objetos que interaccionan entre sí.
- La esencia principal del desarrollo orientado a objetos es el de identificar y organizar conceptos del dominio de la aplicación y no tanto de su representación final en un lenguaje de programación.
- La separación entre el análisis y el diseño es muy pequeña. De hecho, muchas de las técnicas del análisis son usadas en el diseño, pero alcanzando un nivel de detalle mayor. Esta menor separación ayuda a que el paso del análisis al diseño sea mucho más suave que en las metodologías tradicionales.
- En las metodologías orientadas a objetos, a diferencia de lo que ocurre en las tradicionales, se integran las funciones y los datos bajo el concepto de objeto.
- Se eliminan fronteras entre fases debido a la naturaleza iterativa del desarrollo orientado al objeto.
- Aparece una nueva forma de concebir los lenguajes de programación y su uso al incorporarse bibliotecas de clases y otros componentes reutilizables. Fomenta la reutilización en el desarrollo del software.
- Hay un alto grado de iteración y solapamiento, lo que lleva a una forma de trabajo muy dinámica.

Las principales ventajas de las metodologías orientadas a objetos son:

- Son interactivas e incrementales.

- Fácil de dividir el sistema en varios subsistemas independientes.
- Se fomenta la reutilización de componentes.

Algunos ejemplos de metodologías orientadas a objetos son:

- OMT de Jim Rumbaugh.
- OOAD (Object-Oriented Analysis and Design) de Grady Booch.
- OOSE (Object-Oriented Software Engineering) de Jacobson.
- PUDS (el Proceso Unificado de Desarrollo Software) propuesta por Jacobson, Grady Booch y James Rumbaugh. Se caracteriza por estar dirigido por casos de uso, centrado en la arquitectura y por ser iterativo e incremental.
- RUP (Rational Unified Process): Adaptación de IBM de la metodología PUDS.

3.2.2.1 UML

UML (Unified Modeling Language) no es una metodología sino un lenguaje que permite representar distintos elementos. Es un complemento a PUDS y en el que éste se apoya para las labores de representación, pero en sí mismo no es una metodología. Se trata de un estándar de OMG (Object Management Group)

3.2.2.1.1 Diagramas UML

La siguiente figura muestra los distintos diagramas utilizados en UML.

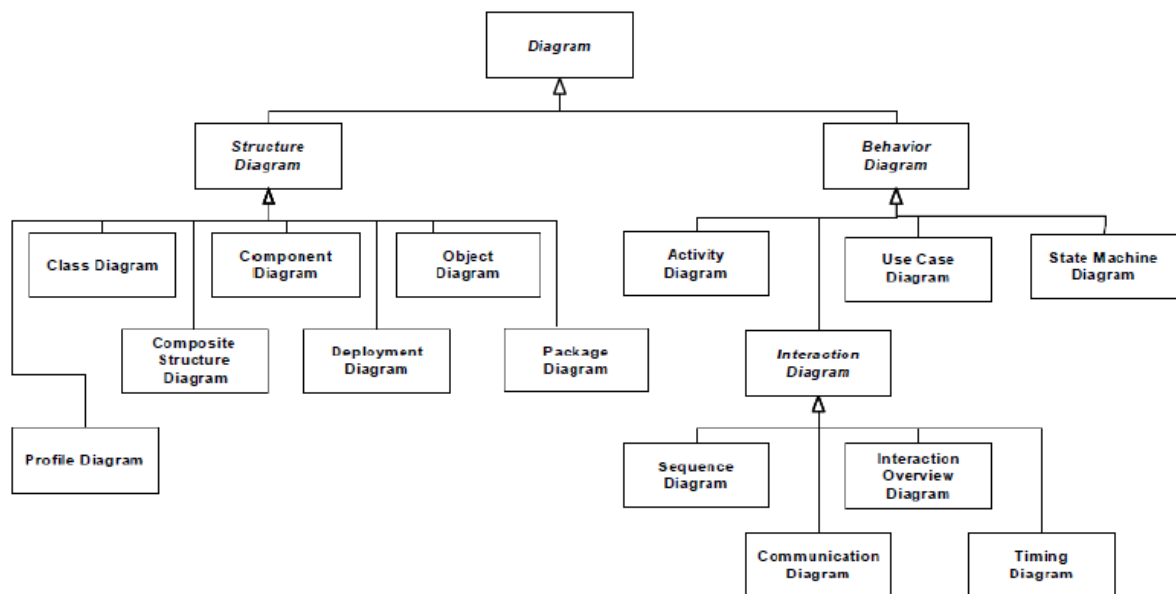


Figura 9 .- Diagramas UML

Aunque no es el objetivo de este tema detallar cada uno de estos diagramas ya que para ello existen temas específicos, se incluye a continuación una breve descripción del objetivo de cada diagrama:

1. **Diagramas estáticos (o de Estructura):** Los diagramas estáticos se centran en la parte estructural del sistema. Son los siguientes:
 - **Diagrama de clases:** Formados por clases y relaciones entre ellas. Las relaciones a su vez pueden ser de asociación, agregación y herencia.

- **Diagrama de objetos:** Similar al diagrama de clases pero para unas instancias concretas, por lo tanto muestra el comportamiento estático en un momento dado.
 - **Diagrama de componentes:** Un componente es una unidad física de implementación con interfaces bien definidas pensada para ser utilizada como parte reemplazable del sistema. Este diagrama muestra los componentes que forman parte del sistema y la relación entre ellos.
 - **Diagrama de despliegue:** Muestra tanto elementos Software como elementos Hardware.
 - **Diagrama de Estructura Compuesta:** Muestra la estructura interna de un elemento.
 - **Diagrama de Perfiles:** Los perfiles se utilizan para adaptar UML (el “metamodelo” que define los diagramas a utilizar) a diferentes propósitos. Introducido en UML 2 (se ha ido modificando y ampliando en las distintas versiones).
2. **Los diagramas dinámicos (o de Comportamiento):** Se centran en el comportamiento del sistema. Son los siguientes:
- **Diagrama de casos de uso:** Representa los casos de uso a los que el sistema debe responder. Muestra actores, casos de uso y relación entre ellos (inclusión y extensión).
 - **Diagrama de estados:** Muestra estados por los que va pasando el sistema o las clases.
 - **Diagrama de actividad:** Representa los procesos de negocio de alto nivel. También puede utilizarse para modelar lógica dentro de un sistema. Es similar a un diagrama de flujo.
 - **Diagramas de interacción:** Muestra cómo colaboran los objetos. Se dividen en:
 - **Diagrama de secuencia:** Muestra de manera cronológica cómo se reciben mensajes entre distintas clases y cómo reaccionan éstas al recibirlos.
 - **Diagrama de comunicación:** Muestra cómo reaccionan las clases cuando reciben mensajes, la finalidad es la misma que la de los diagramas de secuencia pero son más cómodos de realizar, especialmente en la fase de diseño. Tienen una parte estática que viene determinada por el Diagrama de Clases, y una parte dinámica determinada por los mensajes que se reciben.
 - **Diagrama Global de Interacción (Interaction Overview Diagram):** Parte del diagrama de secuencia y añade elementos del diagrama de actividad para describirlo con mayor detalle.
 - **Diagrama de tiempos:** Modelan el comportamiento a lo largo del tiempo. Son más importantes en sistemas de tiempo real.

3.2.2.1.2 Visión de la Arquitectura en UML

La arquitectura queda definida por distintas vistas, que engloban un conjunto de diagramas. Es lo que se llama el modelo de “4+1” Vistas.

1. **Vista Lógica (o de Diseño):** Nos proporciona información de la estructura y la funcionalidad que el sistema da a los usuarios finales. La forman los siguientes diagramas:
 - Diagrama de Clases
 - Diagrama de Comunicación
 - Diagrama de Secuencia

2. **Vista de Desarrollo:** es la más cercana al programador, muestra la estructura de la aplicación. La forman los siguientes diagramas:
 - Diagrama de Componentes
 - Diagrama de Paquetes
3. **Vista de Procesos (o de Interacción):** Nos da información sobre el rendimiento del sistema, la escalabilidad del mismo y la capacidad de procesamiento necesaria. Se utiliza el Diagrama de Actividad para representar esta vista
4. **Vista Física (o de Implementación):** Establece el ensamblado del sistema y la gestión de la configuración. Se utiliza el Diagrama de Despliegue para representar esta vista.
5. **Vista 4+1: Vista de Escenarios (o de Casos de Uso):** Nos facilita información sobre el comportamiento y funcionalidad del sistema. Se encarga de unir y relacionar las otras vistas. Se utiliza el Diagrama de Casos de Uso para representar esta vista.

3.2.3 Metodologías de Tiempo Real

Los sistemas de tiempo real son sistemas que controlan un entorno recibiendo datos, procesándolos y devolviéndolos con la suficiente rapidez como para influir en dicho entorno en ese momento. Un ejemplo de sistema en tiempo real es el piloto automático de un avión, el piloto automático debe tomar medidas del rumbo, velocidad y altitud para mover adecuadamente las superficies de control del avión y ajustar dichas medidas a las que ha especificado el piloto. Así, si la altura es excesiva habrá que hacer descender al avión, o si el rumbo no es correcto habrá que hacerle girar hasta que tome el correcto.

Las metodologías basadas en tiempo real tienen que tener unas características específicas para poder tratar con aspectos específicos del tiempo real. Éstas son:

- Gestión de procesos concurrentes.
- Manejo de interrupciones y prioridades.
- Comunicación y sincronización entre tareas
- Respuesta oportuna ante eventos externos
- Datos continuos o discretos

Las metodologías que se emplean para Tiempo Real se basan en ampliaciones y modificaciones específicas realizadas sobre otras metodologías de uso más general. Así por ejemplo hay metodologías que están basadas en ampliaciones de la notación del análisis estructurado o metodologías orientadas a objetos que han sido convenientemente adaptadas para tratar con los aspectos del tiempo real, como por ejemplo la metodología de Ward y Mellor, que en 1985 proponen la metodología de Desarrollo Estructurado para Sistemas en Tiempo Real, y que se basan en los Diagramas de Flujo de control, definidos por estos autores, por analogía al uso de DFD visto en las metodologías estructuradas.

3.2.4 Metodologías Ágiles

En marzo de 2001 diecisiete críticos de los modelos de mejora del desarrollo de software basados en procesos “tradicionales”, convocados por Kent Beck, se reunieron en Salt Lake City para tratar sobre técnicas y procesos para desarrollar software. Beck había publicado un par de años antes Extreme Programming Explained, libro en el que exponía una nueva metodología denominada Extreme Programming. En la reunión se acuñó el término “Métodos Ágiles” para definir a los métodos que estaban surgiendo como alternativa a las metodologías formales (CMMI, SPICE, ITIL, METRICA, MERISE, RUP) a las que consideraban excesivamente “pesadas” y rígidas por su carácter normativo y fuerte dependencia de planificaciones detalladas previas al desarrollo.

La principal conclusión a la que llegaron fue que los métodos de desarrollo de software actuales no se adecúan bien a los proyectos reales. Las metodologías se basan en aplicar métodos de ingeniería clásica para el desarrollo del software. Todas ellas realizan una planificación al principio del proyecto y generan durante éste una carga burocrática más o menos importante. Fowler critica las metodologías clásicas diciendo que hay tanto que hacer en las metodologías clásicas que retrasan el ritmo de todo el desarrollo.

Las metodologías ágiles o ligeras (como se las denominó al principio), buscan un equilibrio entre no usar ningún método de desarrollo a usar un método de desarrollo demasiado pesado, como hacen las metodologías clásicas.

Los integrantes de la reunión resumieron los principios sobre los que se basan los métodos alternativos en cuatro postulados, lo que ha quedado denominado como Manifiesto Ágil.

En el Manifiesto Ágil, firmado por Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland y Dave Thomas, se expone que:

Estamos poniendo al descubierto mejores métodos para desarrollar software, haciéndolo y ayudando a otros a que lo hagan. Con este trabajo hemos llegado a valorar:

- *A los individuos y su interacción, por encima de los procesos y las herramientas.*
- *El software que funciona, por encima de la documentación exhaustiva.*
- *La colaboración con el cliente, por encima de la negociación contractual.*
- *La respuesta al cambio, por encima del seguimiento de un plan.*

Aunque hay valor en los elementos de la derecha, valoramos más los de la izquierda.

3.2.4.1 Principios del Manifiesto Ágil

El Manifiesto Ágil recoge 12 principios:

1. *Nuestra principal prioridad es satisfacer al cliente a través de la entrega temprana y continua de software de valor.*
2. *Son bienvenidos los requisitos cambiantes, incluso si llegan tarde al desarrollo. Los procesos ágiles se dobligan al cambio como ventaja competitiva para el cliente.*
3. *Entregar con frecuencia software que funcione, en periodos de un par de semanas hasta un par de meses, con preferencia en los periodos breves.*
4. *Las personas del negocio y los desarrolladores deben trabajar juntos de forma cotidiana a través del proyecto.*
5. *Construcción de proyectos en torno a individuos motivados, dándoles la oportunidad y el respaldo que necesitan y procurándoles confianza para que realicen la tarea.*
6. *La forma más eficiente y efectiva de comunicar información de ida y vuelta dentro de un equipo de desarrollo es mediante la conversación cara a cara.*
7. *El software que funciona es la principal medida del progreso.*
8. *Los procesos ágiles promueven el desarrollo sostenido. Los patrocinadores, desarrolladores y usuarios deben mantener un ritmo constante de forma indefinida.*
9. *La atención continua a la excelencia técnica enaltece la agilidad.*
10. *La simplicidad como arte de maximizar la cantidad de trabajo que no se hace, es esencial.*
11. *Las mejores arquitecturas, requisitos y diseños emergen de equipos que se auto-organizan.*
12. *En intervalos regulares, el equipo reflexiona sobre la forma de ser más efectivo y ajusta su conducta en consecuencia.*

Según Fowler, el proceso ágil se caracteriza por las siguientes tres suposiciones sobre los proyectos:

- Resulta difícil predecir los requisitos que persistirán en un sistema y cuales cambiarán.

Igualmente, es difícil prever cómo cambiarán las prioridades del cliente a lo largo del proyecto.

- En muchos sistemas, el diseño y la construcción se intercalan, es decir, se hacen de forma conjunta de forma que los modelos de diseño se prueban mientras se crean.
- El análisis, diseño y construcción, desde el punto de vista de la planificación, no son predecibles, aunque sería deseable.

Dado que hay tanta incertidumbre, el proceso ágil de desarrollo debe ser adaptable, debe irse modificando según se van produciendo cambios. Para poder lograr el avance en el proyecto se debe optar por una estrategia incremental y una retroalimentación por parte del cliente. Las metodologías ágiles utilizan un enfoque llamado “Continuous Delivery” (o desarrollo continuo) en el que las entregas de software, prototipos o partes del sistema, deben entregarse cada poco tiempo, y con garantías de que ese desarrollo puede ponerse en el entorno de producción, para que la adaptación pueda realizarse correctamente a partir de la retroalimentación proporcionada por el cliente al realizar las evaluaciones de las entregas de software.

Los defensores de las metodologías ágiles resaltan la importancia del “factor humano”. Como defienden Cockburn y Highsmith, el desarrollo ágil se centra en los talentos y en las habilidades de los individuos, puesto que el proceso se ajusta a personas y equipos específicos. El punto clave es que el proceso se ajusta a las personas y al equipo, y no al revés. Se han identificado los siguientes rasgos claves que deben tener las personas que trabajen en un equipo ágil:

- **Competencia.** Hace referencia a un talento innato, habilidades específicas relacionadas con el software y conocimiento general del proceso ágil a aplicar.
- **Enfoque Común.** Cada componente del equipo tendrá una tarea encomendada, pero todos deben estar enfocados en la misma meta: Entregar al cliente una nueva versión (un nuevo incremento) de software en el tiempo planificado.
- **Colaboración.** Debe haber colaboración entre los diferentes miembros del equipo, con el cliente y con los gerentes.
- **Habilidad para Tomar Decisiones.** Todo buen equipo de desarrollo de software debe poder controlar su destino, para lo que debe poder tener autonomía para la toma de decisiones de tipo técnico y del proyecto.
- **Capacidad para Resolver Problemas Confusos.** Debido al cambio el equipo se enfrentará a dificultades, debe asumir que el problema que hoy está resolviendo quizás ya no sea el que debe resolver mañana, pero debe asumirlo y las lecciones aprendidas del problema que hoy resuelve tendrán probablemente utilidad en un futuro.
- **Confianza y Respeto Mutuo.** En equipo debe tener una confianza y un respeto de forma que se produzcan sinergias, es decir, como DeMarco dice, “se unan con tanta fuerza, que el todo sea mayor que la suma de la partes”.
- **Organización Propia.** En el contexto de desarrollo ágil, organización propia implica tres factores:
 - El equipo ágil se organiza a sí mismo para el trabajo que hay que realizar.
 - El equipo organiza el proceso que mejor se ajusta a su situación particular.
 - El equipo organiza el programa de trabajo con el que mejor logre alcanzar la entrega del incremento de software.

El equipo debe determinar la cantidad de trabajo que es capaz de hacer en la iteración y se compromete a realizarlo en el tiempo que se ha estimado. Es muy desalentador que otro se comprometa por el equipo, pero por el contrario, es muy motivador para el equipo aceptar las responsabilidades de cumplir los compromisos que él mismo hizo.

3.2.4.2 Principales diferencias entre las metodologías clásicas y las ágiles

- Los métodos ágiles son adaptables en vez de predictivos. Las metodologías clásicas tratan de realizar al principio del proyecto una planificación y se resisten a realizar cambios en dicha planificación (por ejemplo, fijando los requisitos al principio del proyecto y estableciendo el contrato sobre ellos). Sin embargo, los métodos ágiles aceptan el cambio, intentan ser procesos que crecen y se adaptan al cambio.
- Los métodos ágiles están orientados hacia las personas, hacia el “VALOR” no a los procesos. Una metodología clásica debe definir un proceso que pueda ser usado por cualquier persona. Los métodos ágiles afirman que el proceso apoya al equipo de desarrollo en su trabajo pero no podrá hacer que un mal equipo funcione. De hecho, Cockburn defiende que los modelos de proceso tradicionales olvidan las fragilidades de las personas que construyen el software. Los ingenieros no son robots, cada uno tiene su propio estilo, habilidades, creatividad, orden, consistencia y espontaneidad

3.2.4.3 Principales Metodologías Ágiles utilizadas

En los últimos tiempos han surgidos distintas metodologías que siguen los principios del Manifiesto Ágil aplicando distintos métodos, ejemplos de estas metodologías son las siguientes:

- Adaptive Software Development (ASD).
- Agile Unified Process (AUP).
- Behaviour Development Driven BDD
- Crystal Clear.
- Essential Unified Process (EssUP).
- Feature Driven Development - FDD
- Lean Software Development - LSD
- Kanban.
- Open Unified Process (OpenUP).
- Programación Extrema (XP).
- Método de desarrollo de sistemas dinámicos (DSDM).
- Scrum
- Scrumban
- Test Development Driven (TDD)

En la siguiente gráfica se puede ver el grado de utilización de las distintas variantes de Metodologías Ágiles:

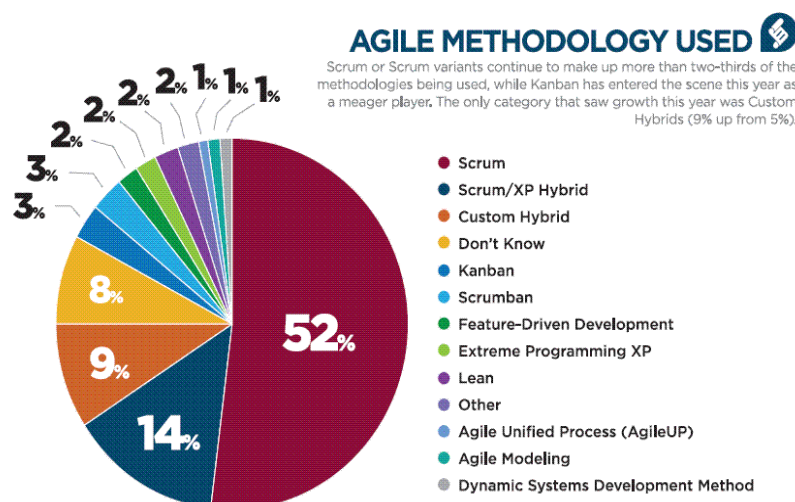


Figura 10 .- Gráfico de uso de metodología Ágil

Como se puede apreciar Scrum es la más utilizada, habiendo surgido distintas variedades o adaptaciones de Scrum. A continuación se describirán las principales metodologías.

3.2.4.4 Scrum

Scrum es un marco de trabajo para la gestión y desarrollo de software basada en un proceso iterativo e incremental utilizado comúnmente en entornos basados en el desarrollo ágil de software. Aunque está enfocado a la gestión de procesos de desarrollo de software, puede ser utilizado en equipos de mantenimiento de software, o de gestión de programas: Scrum de Scrums.

Scrum es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar colaborativamente, en equipo, y obtener el mejor resultado posible de un proyecto. Estas prácticas se apoyan unas a otras y su selección tiene origen en un estudio de la manera de trabajar de equipos altamente productivos.

En Scrum se realizan entregas parciales y regulares del producto final, priorizadas por el beneficio que aportan al receptor del proyecto. Por ello, Scrum está especialmente indicado para proyectos en entornos complejos, donde se necesita obtener resultados pronto, donde los requisitos son cambiantes o poco definidos, donde la innovación, la competitividad, la flexibilidad y la productividad son fundamentales.

Scrum también se utiliza para resolver situaciones en que no se está entregando al cliente lo que necesita, cuando las entregas se alargan demasiado, los costes se disparan o la calidad no es aceptable, cuando se necesita capacidad de reacción ante la competencia, cuando la moral de los equipos es baja y la rotación alta, cuando es necesario identificar y solucionar ineficiencias sistemáticamente o cuando se quiere trabajar utilizando un proceso especializado en el desarrollo de producto.

En el desarrollo de software Scrum está considerado como modelo ágil por la **Agile Alliance**.

3.2.4.4.1 Características Generales de Scrum

Scrum es un modelo de referencia que define un conjunto de prácticas y roles, y que puede tomarse como punto de partida para definir el proceso de desarrollo que se ejecutará durante un proyecto.

Durante cada sprint, un periodo entre 15 y 30 días (la magnitud es definida por el equipo), el equipo crea un incremento de software potencialmente utilizable. El conjunto de características que forma parte de cada sprint viene del Product Backlog, que es un conjunto de requisitos de alto nivel priorizados que definen el trabajo a realizar. Los elementos del Product Backlog que forman parte del sprint se determinan durante la reunión de Sprint Planning. Durante esta reunión, el Product Owner identifica los elementos del Product Backlog que quiere ver completados y los hace del conocimiento del equipo. Entonces, el equipo determina la cantidad de ese trabajo que puede comprometerse a completar durante el siguiente sprint.

Durante el sprint, nadie puede cambiar el Sprint Backlog, lo que significa que los requisitos están congelados durante el sprint.

Scrum permite la creación de equipos autoorganizados impulsando la co-localización de todos los miembros del equipo, y la comunicación verbal entre todos los miembros y disciplinas involucrados en el proyecto.

Un principio clave de Scrum es el reconocimiento de que durante un proyecto los clientes pueden cambiar de idea sobre lo que quieren y necesitan (a menudo llamado requirements churn), y que los desafíos impredecibles no pueden ser fácilmente enfrentados de una forma predictiva y planificada. Por lo tanto, Scrum adopta una aproximación pragmática, aceptando que el problema no puede ser completamente entendido o definido, y centrándose en maximizar la capacidad del equipo de entregar rápidamente y responder a requisitos emergentes.

Existen varias implementaciones de sistemas para gestionar el proceso de Scrum, que van desde notas amarillas "post-it" y pizarras hasta paquetes de software. Una de las mayores ventajas de Scrum es que es muy fácil de aprender, y requiere muy poco esfuerzo para comenzarse a utilizar.

3.2.4.4.2 Roles en Scrum

En Scrum se definen varios roles, estos están divididos en dos grupos: cerdos y gallinas. El nombre de los grupos está inspirado en la siguiente frase:

En un plato de huevos con tocino el cerdo está comprometido, la gallina sólo está involucrada.

- Los 'cerdos' están comprometidos a desarrollar el software de forma regular y frecuente
- Las 'gallinas' sólo están interesados en el proyecto. Si falla, fueron los cerdos los que se comprometieron.

Las necesidades, deseos, ideas e influencias de los roles 'gallina' se tienen en cuenta, pero no de forma que pueda afectar, distorsionar o entorpecer el proyecto Scrum.

- Roles "Cerdo"

- **Product Owner:** El Propietario del producto representa la voz del cliente. Se asegura de que el equipo Scrum trabaja de forma adecuada desde la perspectiva del negocio. El Product Owner escribe historias de usuario, las prioriza, y las coloca en el Product Backlog.
- **ScrumMaster (o Facilitador):** El Scrum es facilitado por un ScrumMaster, cuyo trabajo primario es eliminar los obstáculos que impiden que el equipo alcance el objetivo del sprint. El ScrumMaster no es el líder del equipo (porque ellos se auto-organizan), sino que actúa como una protección entre el equipo y cualquier influencia que le distraiga. El ScrumMaster se asegura de que el proceso Scrum se utiliza como es debido. El ScrumMaster es el que hace que las reglas se cumplan.
- **ScrumTeam o Equipo:** El equipo tiene la responsabilidad de entregar el producto. Un pequeño equipo de 5 a 9 personas con las habilidades transversales necesarias para realizar el trabajo (diseñador, desarrollador, etc).

- Roles "Gallina"

Los roles gallina en realidad no son parte del proceso Scrum, pero deben tenerse en cuenta. Un aspecto importante de una aproximación ágil es la práctica de involucrar en el proceso a los usuarios, expertos del negocio y otros interesados (stakeholders). Es importante que esa gente participe y entregue retroalimentación con respecto a la salida del proceso a fin de revisar y planear cada sprint.

- **Usuarios:** Es el destinatario final del producto. Debe tenerse en cuenta porque si el software no es usado es como si nunca hubiera sido desarrollado.
- **Stakeholders** (Clientes, Proveedores, Inversores): Se refiere a la gente que hace posible el proyecto y para quienes el proyecto producirá el beneficio acordado que lo justifica. Sólo participan directamente durante las revisiones del sprint.
- **Managers:** Es la gente que establece el ambiente para el desarrollo del producto.

3.2.4.4.3 Reuniones en Scrum

- Reunión de Planificación del Sprint (Sprint Planning Meeting)

Al inicio de cada Sprint (cada 15 o 30 días), se realiza una "Reunión de Planificación del Sprint".

El **Product Owner** es quien prioriza sobre el product backlog.

Objetivos de la reunión: (Ocho horas como límite)

- Seleccionar qué trabajo se hará
- Preparar, con el equipo completo, el Sprint Backlog que detalla el tiempo que llevará hacer el trabajo.
- Identificar y comunicar cuánto del trabajo es probable que se realice durante el actual Sprint

- Daily Scrum (Sincronizaciones Diarias o Daily Standup)

Cada día de un sprint, se realiza una reunión sobre el estado del proyecto. SCRUM tiene unas reglas o directrices específicas:

- La reunión comienza puntualmente a su hora. A menudo hay “castigos” acordados por el equipo para quien llegue tarde (por ejemplo: dinero, flexiones, llevar colgando una gallina de plástico del cuello, pagar los cafés, etc)
- Todos son bienvenidos, pero sólo los “roles cerdos” pueden hablar.
- La reunión tiene una duración fija de 15 minutos, de forma independiente del tamaño del equipo. No es para resolver in-situ los problemas sino para identificarlos.
- Todos los asistentes deben mantenerse de pie (esto ayuda a mantener la reunión corta)
- La reunión debe ocurrir en la misma ubicación y a la misma hora todos los días.

Durante la reunión, cada miembro del equipo contesta a tres preguntas:

- ¿Qué has hecho desde ayer?
- ¿Qué es lo que estás planeando hacer hoy?
- ¿Has tenido algún problema que te haya impedido alcanzar tu objetivo? (Es el papel del ScrumMaster recordar estos impedimentos).

- **Scrum de Scrum (Reuniones de Equipos)**

- Se trata de una reunión de coordinación que se aplica en proyectos de tamaño medio o grande.
- Se realiza cada día después del “Daily Scrum”
- Estas reuniones permiten a los grupos de equipos discutir su trabajo, enfocándose especialmente en áreas de solapamiento e integración.
- Asiste una persona asignada por cada equipo.

La agenda será la misma que la del Daily Scrum, añadiendo además las siguientes cuatro preguntas:

- ¿Qué ha hecho tu equipo desde nuestra última reunión?
- ¿Qué hará tu equipo antes que nos volvamos a reunir?
- ¿Hay algo que demora o estorba a tu equipo?
- ¿Estás a punto de poner algo en el camino del otro equipo?

- **Reunión de Revisión del Sprint (Sprint Review Meeting)**

Se lleva a cabo al final de cada Sprint. Tiene un límite de 4 horas:

- Revisar el trabajo que fue completado y no completado
- Presentar el trabajo completado a los interesados (alias “demo”): El trabajo incompleto no puede ser demostrado

- **Retrospectiva del Sprint (Sprint Retrospective)**

Después de cada sprint, se lleva a cabo una retrospectiva (revisión) del sprint, en la cual todos los miembros del equipo dejan sus impresiones sobre el sprint recién superado. El propósito de la retrospectiva es realizar una mejora continua del proceso.

Esta reunión tiene un tiempo fijo de cuatro horas máximo.

3.2.4.4.4 Documentos

- **Product backlog (Pila del producto)**

El product backlog o Pila del producto es un documento de alto nivel para todo el proyecto.

Contiene descripciones genéricas de todos los requisitos, funcionalidades deseables, etc. priorizadas según su retorno sobre la inversión (ROI). Primero se realizarán aquellos requisitos más interesantes.

- Es lo que va a ser construido.
- Lo gestiona el product owner, quien añadirá los requisitos a petición de los stakeholders.
- Contiene estimaciones grosso modo, tanto del valor para el negocio, como del esfuerzo de desarrollo requerido. Esta estimación ayuda al product owner a ajustar la línea temporal y, de manera limitada, la prioridad de las diferentes tareas.

Por ejemplo, si dos características tienen el mismo valor de negocio la que requiera menos tiempo de desarrollo tendrá probablemente más prioridad, debido a que su ROI será más alto.

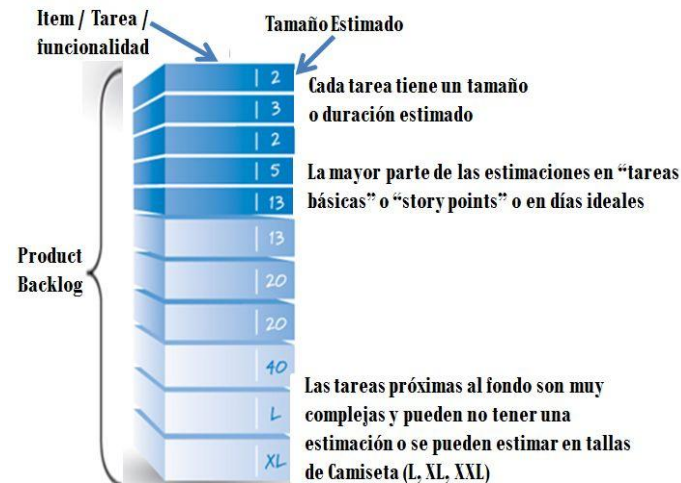


Figura 11.- Product Backlog priorizado y con estimación de duración de las tareas

- Sprint backlog (Pila del sprint)

El sprint backlog o Pila del sprint es un documento detallado donde se describe el cómo el equipo va a implementar los requisitos durante el siguiente sprint. Las tareas se dividen en horas con ninguna tarea de duración superior a 16 horas. Si una tarea es mayor de 16 horas, deberá ser dividida en otras menores. Las tareas en el sprint backlog nunca son asignadas, son tomadas por los miembros del equipo del modo que les parezca oportuno.

- Burn down Chart (Gráfica de "quemado")

La burn down chart es una gráfica mostrada públicamente que mide la cantidad de requisitos en el Backlog del proyecto pendientes al comienzo de cada Sprint. Dibujando una línea que conecte los puntos de todos los Sprints completados, podremos ver el progreso del proyecto. Lo normal es que esta línea sea descendente (en casos en que todo va bien en el sentido de que los requisitos están bien definidos desde el principio y no varían nunca) hasta llegar al eje horizontal, momento en el cual el proyecto se ha terminado (no hay más requisitos pendientes de ser completados en el Backlog). Si durante el proceso se añaden nuevos requisitos la recta tendrá pendiente ascendente en determinados segmentos, y si se modifican algunos requisitos la pendiente variará o incluso valdrá cero en algunos tramos.

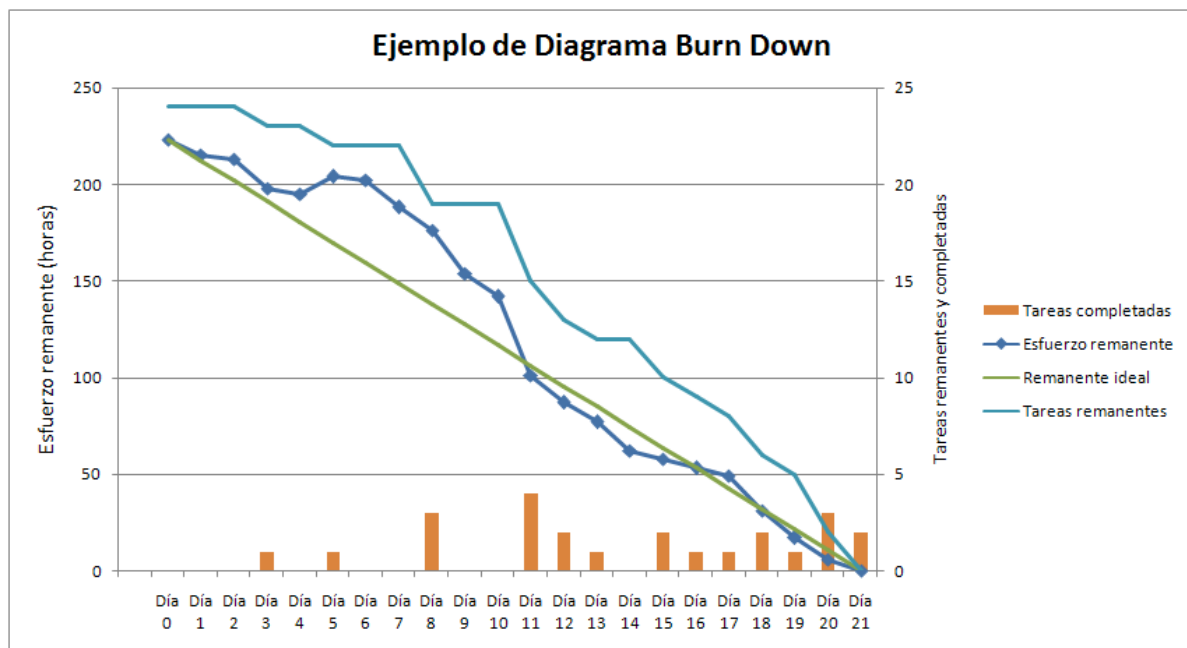


Figura 12.- Gráfica de Quemado (Burn down Chart)

3.2.4.5 Kanban

El origen de la metodología Kanban no está en los proyectos de IT sino en los procesos de producción “just-in-time” (JIT) ideados por Toyota, en los que se utilizaban tarjetas para identificar necesidades de material en la cadena de producción.

Kanban es una palabra japonesa que significa “tarjetas visuales”.

3.2.4.5.1 Los principios de la metodología Kanban

La metodología Kanban se basa en una serie de principios que la diferencian del resto de metodologías conocidas como ágiles:

1. **Visualizar el flujo de trabajo:** Kanban utiliza las pizarras para visualizar el trabajo y el proceso por el que pasa.

La siguiente figura muestra un ejemplo de pizarra Kanban:

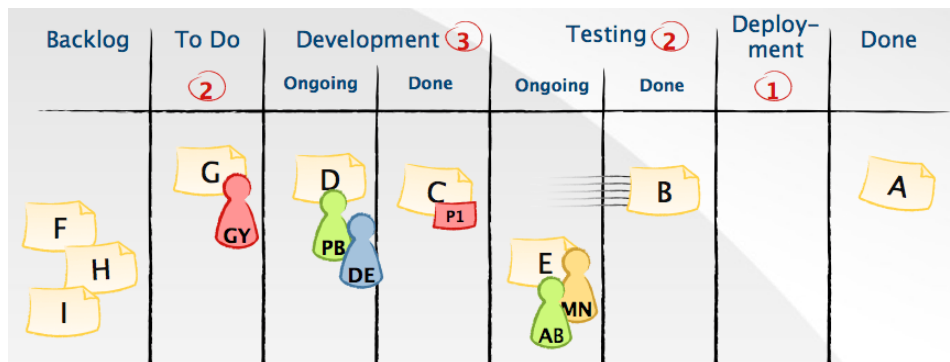


Figura 13 .-Pizara Kanban

La pizarra puede ser física o virtual. Las tareas se representan con tarjetas que van avanzando en el tablero hasta ser completadas.

2. **Limitar el trabajo en curso (WIP – Work In Progress):** Se considera que si se establecen límites al trabajo en curso, de manera que no se empiecen tareas cuando se alcanza ese límite se puede mejorar el flujo de trabajo y reducir tiempos de espera, mejorando la calidad y realizando entregas más frecuentes.

En Kanban el límite se establece para cada una de las columnas de la pizarra, si observamos la imagen anterior se puede ver que en cada columna aparece un número: ese número representa el número máximo de tareas de ese tipo que puede haber. Ésta es una diferencia fundamental con Scrum, aunque en Scrum también se utilizan pizarras similares no hay establecido un número máximo de tarjetas en cada columna.

Una de las frases que suele utilizarse para definir esta metodología es “Stop starting and start finishing”: el principal objetivo es terminar tareas, no mantener muchas empezadas.

3. **Establecer políticas explícitas de calidad:** las políticas deberían ser pocas, sencillas y bien definidas. Ejemplos de políticas son: Límites de trabajo en curso o la definición de cuándo se considera que algo está completo.
4. **Medir y gestionar el flujo de trabajo:** La principal métrica para Kanban es el tiempo de ciclo, definido como el tiempo que tarda una tarjeta en recorrer el tablero. Se trata de un valor que se debe optimizar. En Kanban el tiempo de ciclo no es fijo, al contrario que en Scrum donde los sprints tiene una longitud fija.

Uno de los principales beneficios de esta metodología es que permite identificar fácilmente cuellos de botella, esto se ve de manera muy gráfica en las pizarras, cuando una de las fases del flujo de trabajo se encuentra saturada de trabajo.

5. **Realizar ciclos de retroalimentación:** El objetivo de estos ciclos es la mejora continua del proceso. En la metodología se establecen reuniones periódicas de revisión. Kanban está dentro de la estrategia Kaizen: la mejora continua y continuada.
6. **Mejorar colaborando y evolucionar experimentando:** Se sugiere empezar con el proceso tal y como lo tenemos e ir mejorándolo a medida que se va trabajando en él, en lugar de intentar llegar al objetivo directamente. De esta forma se obtienen resultados antes.

3.2.4.6 XP - EXTREME PROGRAMMING (PROGRAMACIÓN EXTREMA)

XP es una metodología ágil centrada en potenciar las relaciones interpersonales como clave para el éxito en desarrollo de software, promoviendo el trabajo en equipo, preocupándose por el aprendizaje de los desarrolladores, y propiciando un buen clima de trabajo. XP se basa en realimentación continua entre el cliente y el equipo de desarrollo, comunicación fluida entre todos los participantes, simplicidad en las soluciones implementadas y coraje para enfrentar los cambios. XP se define como especialmente adecuada para proyectos con requisitos imprecisos y muy cambiantes, y donde existe un alto riesgo técnico.

Tiene éxito cuando el cliente selecciona el valor de negocio a implementar basado en la habilidad del equipo para medir la funcionalidad que puede entregar a través del tiempo. El mayor objetivo de XP es la reducción de costes mediante una adecuada estimación de la velocidad del proyecto.

Las historias de usuario son la técnica utilizada en XP para especificar los requisitos del software. Se trata de tarjetas de papel en las cuales el cliente describe brevemente las características que el sistema debe poseer, sean requisitos funcionales o no funcionales (tal y como se vio previamente en este tema).

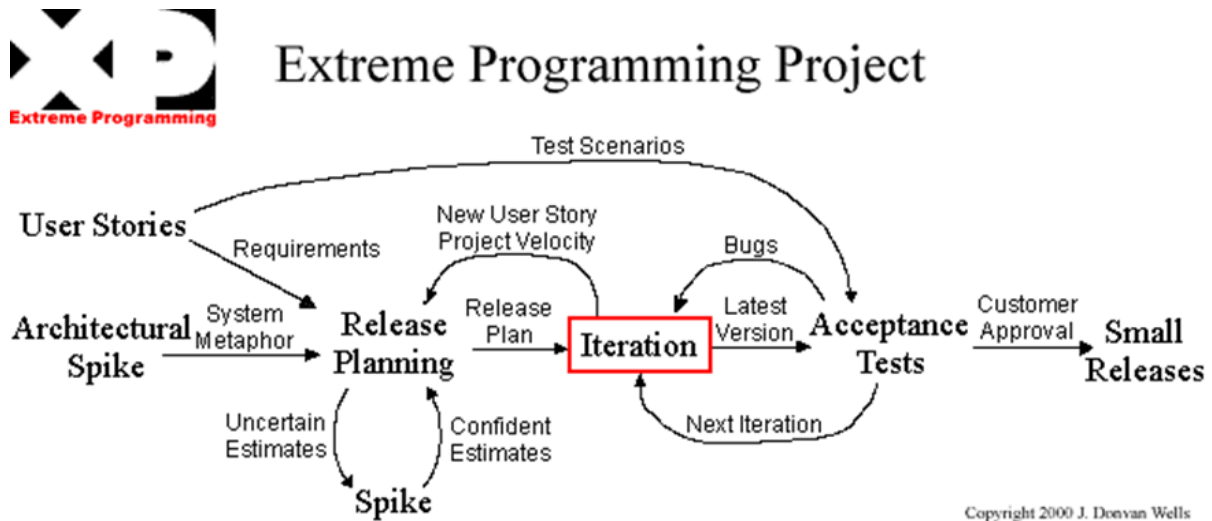


Figura 14 .-Flujo de trabajo en la Metodología Ágil Extreme Programming

3.2.4.6.1 Ciclo de desarrollo:

1. El cliente define el valor de negocio a implementar.
2. El programador estima el esfuerzo necesario para su implementación.
3. El cliente selecciona qué construir, de acuerdo con sus prioridades y las restricciones de tiempo.
4. El programador construye ese valor de negocio. Se calcula la velocidad de proyecto.
5. Vuelve al paso 1

3.2.4.6.2 Roles XP

- Programador
- Cliente

- Encargado de pruebas (Tester)
- Encargado de seguimiento (Tracker)
- Entrenador (Coach)
- Consultor
- Gestor (Big boss)

3.2.4.6.3 Características más significativas de la programación extrema

- El juego de la planificación: los técnicos estiman tiempo, los clientes priorizan funcionalidad y valor
- Entregas pequeñas.
- Metáfora: No se enfatiza la definición temprana de una arquitectura estable, se asume evolutiva. La metáfora es una historia compartida entre el cliente y el equipo de desarrollo que describe cómo debería funcionar el sistema
- Diseño simple.
- Las pruebas unitarias son establecidas antes de escribir el código y son ejecutadas constantemente.
- Refactorización (Refactoring): Es la reestructuración del código fuente sin cambiar el comportamiento externo
- Programación en parejas.
- Propiedad colectiva del código: Rotación entre programadores.
- Integración continua.
- 40 horas por semana: No se justifica el exceso de trabajo continuado.
- Cliente in-situ.
- Estándares de programación: XP enfatiza la comunicación de los programadores a través del código.

3.2.4.6.4 El ciclo de vida ideal de XP

El ciclo de vida ideal de XP consta de seis fases:

1. **Exploración:** Los clientes plantean a grandes rasgos las historias de usuario que son de interés para la primera entrega del producto.
2. **Planificación de la entrega (Release):** El cliente establece la prioridad de cada historia de usuario, y los programadores realizan una estimación del esfuerzo necesario de cada una de ellas. Al planificar se obtiene el número de iteraciones necesarias para su implementación.
3. **Iteraciones:** Esta fase incluye varias iteraciones sobre el sistema antes de ser entregado. El Plan de Entrega está compuesto por iteraciones de no más de tres semanas. En la primera iteración se puede intentar establecer una arquitectura del sistema que pueda ser utilizada durante el resto del proyecto. Al final de la última iteración el sistema estará listo para entrar en producción.
4. **Producción:** La fase de producción requiere de pruebas adicionales y revisiones de rendimiento antes de que el sistema sea trasladado al entorno del cliente. Las ideas que han sido propuestas y las sugerencias son documentadas para su posterior implementación (por ejemplo, durante la fase de mantenimiento).
5. **Mantenimiento:** Mientras la primera versión se encuentra en producción, el proyecto XP debe mantener el sistema en funcionamiento al mismo tiempo que desarrolla nuevas iteraciones. Para realizar esto se requiere de tareas de soporte para el cliente.

- 6. Finalización del proyecto:** Es cuando el cliente no tiene más historias para ser incluidas en el sistema. Se genera la documentación final del sistema y no se realizan más cambios en la arquitectura. Se refiere generalmente como la “Muerte del proyecto”.

3.2.4.6.5 Valores en la Programación Extrema (XP)

Esta metodología está basada en cuatro valores:

- **Simplicidad:** de diseño, código y documentación
- **Comunicación:** la comunicación con el cliente es fluida ya que el cliente forma parte del equipo de desarrollo. El cliente decide qué características tienen prioridad y siempre debe estar disponible para solucionar dudas
- **Retroalimentación (feedback).** Al estar el cliente integrado en el proyecto, su opinión sobre el estado del proyecto se conoce en tiempo real.
- **Coraje:** Se requiere coraje para implementar las características que el cliente quiere en este momento, sin caer en la tentación de optar por un enfoque más flexible que permita futuras modificaciones

Posteriormente se añadió un quinto valor:

- **Respeto:** los miembros del equipo respetan el trabajo del resto no haciendo menos a otros, sino orientándolos a realizarlo mejor, obteniendo como resultado una mejor autoestima en el equipo y elevando el ritmo de producción en el equipo.

3.2.4.7 Feature Driven Development

FDD o desarrollo **basado en funcionalidades (o rasgos)** es un enfoque ágil para el desarrollo de sistemas. Fue desarrollado por Jeff De Luca y el viejo gurú de la Orientación a Objetos Peter Coad. Como las otras metodologías ágiles, se centra en iteraciones cortas que entregan funcionalidad tangible. Dicho enfoque no hace énfasis en la obtención de los requisitos sino en cómo se realizan las fases de diseño y construcción. Sin embargo, fue diseñado para trabajar con otras actividades de desarrollo de software y no requiere la utilización de ningún modelo de proceso específico. Además, hace énfasis en aspectos de calidad durante todo el proceso e incluye monitorización permanente del avance del proyecto. Al contrario de otras metodologías, FDD afirma ser conveniente para el desarrollo de sistemas críticos.

Los desarrolladores se agrupan en dos tipos:

- Dueños de clases
- Programadores jefe

Los programadores jefe son los desarrolladores más experimentados. A ellos se les asignan funcionalidades a construir, sin embargo, ellos no las construyen sino que identifican qué clases se involucran en la implantación de una funcionalidad, seleccionan y juntan a los dueños de dichas clases para que formen un equipo para desarrollarla. El programador jefe actúa como el coordinador, diseñador líder y mentor, mientras los dueños de clases hacen gran parte de la codificación del rasgo.

3.2.4.7.1 El Proceso

El proceso consiste de cinco fases secuenciales durante las cuales el diseño y la construcción del sistema se llevan a cabo.

La parte iterativa del proceso de FDD (Diseño y Construcción) soporta un desarrollo ágil, con adaptaciones rápidas para cambios de último momento en los requisitos. El proceso consta de cinco fases:

1) Develop an Overall Model (desarrollar modelo general)

Cuando esta fase comienza, los expertos del dominio ya tienen una idea del contexto y los requisitos del sistema. Es probable que el documento de especificación de requisitos ya exista. Sin embargo, FDD no hace énfasis en la recolección y administración de los requisitos. Los expertos del dominio presentan un informe llamado walkthrough en el cual los miembros del equipo y el Chief Architect son informados a través de una descripción de alto nivel del sistema.

El dominio global es dividido en diferentes áreas y se realiza un walkthrough detallado para cada una de dichas áreas por parte de los expertos del dominio. Después de cada walkthrough, un grupo de desarrolladores realiza un modelo de objetos para el área del dominio. Además, el equipo de desarrollo discute y decide cual es el modelo de objetos más apropiado para cada área del dominio. Simultáneamente, el modelo global del sistema es construido.

2) Build a Feature List (construir lista de Funcionalidades)

Los walkthroughs, el modelo de objetos y los requisitos existentes ofrecen una buena base para construir una lista de funcionalidades que resuma la funcionalidad del sistema a ser desarrollado. En dicha lista, el equipo de desarrolladores presenta cada una de las funcionalidades evaluadas por el cliente. Las funcionalidades son presentadas por cada área del dominio. Dicha lista es dividida en subconjuntos en base a la funcionalidad. La lista de Funcionalidades es revisada por los usuarios y sponsors (Patrocinadores o financiadores) del sistema para su validación y aprobación.

3) Plan by Feature (planificar en función de las Funcionalidades)

En esta etapa se incluye la creación de un plan de alto nivel, en el cual la lista de funcionalidades se ordena en base a la prioridad y a la dependencia entre cada funcionalidad. Además, las clases identificadas en la primera etapa son asignadas a cada programador.

4) Iteración: Design and Build by Feature (diseñar y construir por Funcionalidades)

Un conjunto de funcionalidades se selecciona de la lista de Funcionalidades. El diseño y construcción de la funcionalidad es un proceso iterativo durante el cual las Funcionalidades seleccionadas se generan. Una iteración puede llevar desde unos pocos días a un máximo de dos semanas.

Este proceso iterativo incluye tareas como

- Inspección del diseño
- Codificación
- Testing unitario – Pruebas unitarias
- Integración e inspección del código.

Una vez que la iteración llega a su fin se realiza un main build de la funcionalidad en el cual se integra la funcionalidad. Dicho main build se realiza mientras la siguiente iteración comienza.

La siguiente figura muestra este proceso:

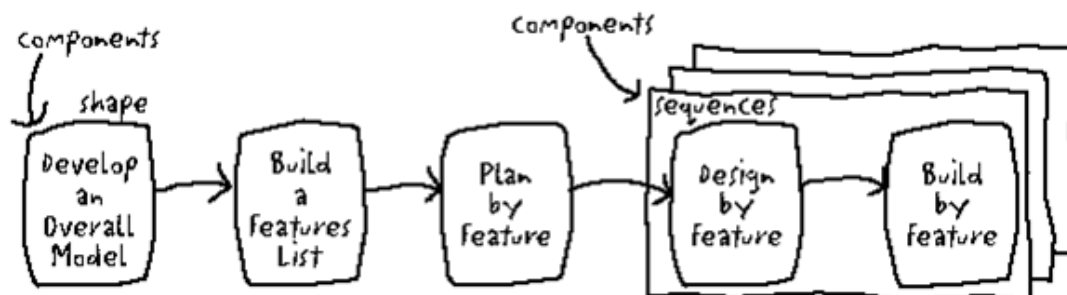


Figura 15.- Proceso FDD dividido en cinco fases

3.2.4.7.2 Roles y Responsabilidades.

El FDD clasifica sus roles en las siguientes categorías:

- **Roles principales:** Roles responsables del desarrollo del proyecto, entre los que se encuentran el Project manager, el Chief Architect (Arquitecto jefe), el Development Manager, el Chief Programmer, el Class Owner y los expertos de dominio.
- **Roles de soporte o complementarios:** Realizan labores de soporte. Entre ellos están el Release Manager (responsable de entregas), el Language lawyer o language guru (experto

IT), el build engineer (ingeniero de entregas), el toolsmith (para herramientas de soporte) y el system administrator (administrador de sistemas)

- **Roles adicionales:** Entre ellos se encuentran los deployers (desplegadores), los technical writers (redactores técnicos para preparar documentación de usuario) y los testers (pruebas)

Un miembro de un equipo puede ejercer varios roles y un rol puede ser compartido por varias personas.

3.2.4.8 TDD - Test Driven Development

TDD o Desarrollo guiado por pruebas es una metodología ágil propuesta por Ken Beck y Dave Astels en 2003. Promueve los test unitarios de cada una de las nuevas funcionalidades que se vayan a realizar.

Involucra otras dos prácticas de ingeniería de software:

- Escribir las pruebas primero (Test First Development)
- Refactorización (Refactoring).

Para escribir las pruebas generalmente se utilizan las pruebas unitarias (“unit test” en inglés). El propósito del desarrollo guiado por pruebas es lograr un código limpio que funcione. La idea es que los requisitos sean traducidos a pruebas, de este modo, cuando las pruebas pasen se garantizará que el software cumple con los requisitos que se han establecido, o dicho de otro modo, cuando se definen los requisitos se piensa “lo primero” en cómo puedo verificar cada requisito, y diseño las pruebas con ese objetivo antes de implementar el código.

Para que funcione el desarrollo guiado por pruebas, el sistema que se programa tiene que ser lo suficientemente flexible como para permitir que sea probado automáticamente.

Las pruebas deben ser lo suficientemente pequeñas para permitir determinar unívocamente si el código probado pasa o no la verificación que ésta le impone.

Vinculado al concepto de “Desarrollo guiado por pruebas” está el concepto de **integración continua**: integrar continuamente consiste en hacer integraciones automáticas (compilación, despliegue y ejecución de pruebas) de un proyecto lo más a menudo posible para así poder detectar y notificar fallos cuanto antes. Esto permite ejecutar todas las baterías de pruebas y así descubrir si nuestra última versión es compatible con el resto del sistema, si tiene algún fallo anteriormente identificado y/o corregido, y en general conocer cuál es el grado de cobertura de requisitos verificados y la fiabilidad de nuestro proyecto. Es recomendable y mucho menos costoso corregir pequeños problemas continuamente que enfrentarse a problemas “*enormes*” cerca de la fecha de entrega fijada. Realmente el concepto de integración continua es mucho más amplio y puede aplicar con éxito en el resto de metodologías ágiles, también sería posible hacerlo en metodologías tradicionales con características evolutivas y/o incrementales. En el mercado existen distintas herramientas que facilitan este enfoque, no es el objetivo de este tema profundizar en estas técnicas, únicamente se menciona por su relación con la metodología de desarrollo guiado por pruebas.

3.2.4.8.1 Ciclo de desarrollo conducido por pruebas

Partiendo de una lista de requisitos a implementar se ejecuta el siguiente ciclo por cada requisito:

- 1) **Elegir un requisito:** Se elige de una lista el requisito que se cree que nos dará mayor conocimiento del problema y que a la vez sea fácilmente implementable.
- 2) **Escribir una prueba:** Se comienza escribiendo una prueba para el requisito. Para ello el programador debe entender claramente las especificaciones y los requisitos de la funcionalidad que está por implementar. Este paso fuerza al programador a tomar la perspectiva de un cliente considerando el código a través de sus interfaces.
- 3) **Verificar que la prueba falla:** Si la prueba no falla es porque el requisito ya estaba implementado o porque la prueba es errónea.

- 4) **Escribir la implementación:** Escribir el código más sencillo que haga que la prueba funcione. Se usa la metáfora "Déjelo simple" ("Keep It Simple, Stupid" (KISS)).
- 5) **Ejecutar las pruebas automatizadas:** Verificar si todo el conjunto de pruebas funciona correctamente.
- 6) **Eliminación de duplicación:** El paso final es la refactorización, que se utilizará principalmente para eliminar código duplicado. Se hacen de a una vez un pequeño cambio y luego se corren las pruebas hasta que funcionen.
- 7) **Actualización de la lista de requisitos:** Se actualiza la lista de requisitos tachando el requisito implementado. Asimismo se agregan requisitos que se hayan visto como necesarios durante este ciclo y se agregan requisitos de diseño (P. ej que una funcionalidad esté desacoplada de otra).

3.2.4.9 Escalado Ágil

Las metodologías ágiles, tal y como hemos visto en los apartados anteriores, se centran en cómo aportar valor con entregas frecuentes estando su ámbito de aplicación enfocado al equipo.

El escalado ágil surge para cubrir las necesidades, a nivel de organización, para poder implementar una estrategia ágil. Están basados en frameworks cuyo objetivo es el de tratar las dependencias entre los distintos equipos (que a su vez trabajarán con alguna metodología ágil como pueda ser Scrum o XP).

Este tipo de frameworks sólo son necesarios en organizaciones que desean utilizar metodologías ágiles pero a la vez requieren su gestión conjunta por tratarse de proyectos de gran complejidad vinculados a diversas áreas y con múltiples intervinientes,

Los principales frameworks Scaled Agile, o de escalado ágil, son los siguientes:

- **Nexus:** Desarrollado por Ken Schwaber y Scrum.org. Está basado en Scrum, su principal objetivo es reducir dependencias entre equipos, para ello incluye una nueva reunión llamada Nexus Daily Scrum destinada a identificar esas dependencias y los posibles problemas de integración.
- **LeSS (Large Scale Scrum):** Creado por Craig Larman y Bas Vodde. También está basado en Scrum. Es muy flexible, permite el escalado de Scrum en función del tamaño de la organización.
- **SAFe (Scaled Agile Framework):** Creado por Dean Leffingwell, permite incorporar metodologías ágiles como Scrum, XP y Kanban. Se basa en entregas frecuentes, a nivel de organización, dentro de un calendario regular y predecible, muy basado en el concepto de "Continuous Delivery". Tiene un coste de implementación más elevado que los anteriores.

4 Conclusiones

Tal y como se ha descrito a lo largo del presente tema, la construcción de un producto software es un proceso complejo en el que es básico hacer una labor de análisis inicial donde se recopilen, de manera priorizada, las características o requisitos que nuestro sistema ha de cumplir.

Dada la importancia de este primer paso de establecimiento de requisitos, la ingeniería del software ha buscado distintas técnicas que permiten tanto su captura, como su gestión y discusión con todos los implicados en el proyecto. No tiene menos importancia la gestión del cambio de los requisitos, aspecto que se ha descubierto inherente al propio proceso de construcción de software, de esta forma, en la ingeniería del software existen distintas aproximaciones para manejar estos cambios, manteniendo la trazabilidad de los requisitos y el control de versiones tanto de código como de documentación asociada.

Además de lo anterior, el propio proceso de construcción ha sido objeto de estudio, surgiendo distintas metodologías que han pretendido hacer de esta labor un proceso de ingeniería con unos resultados predecibles. La experiencia en el desarrollo y mantenimiento de sistemas de información ha dado lugar a una evolución en las metodologías, desde las llamadas metodologías clásicas a las metodologías ágiles, basadas en entregas más rápidas y en una mejor respuesta al cambio de requisitos. Cabe destacar que esto no significa que unas metodologías sean mejores que otras, cada una tiene sus propias características, la elección dependerá, en gran medida, de la organización donde se deba implantar, por ello es importante conocer las diferentes propuestas, de manera que se pueda escoger la que mejor se adapte a nuestras necesidades.

GLOSARIO

ASD: Adaptive Software Development
AUP: Agile Unified Process
BDD: Behaviour Development Driven
CASE: Computer-Aided Software Engineering
CLP: Construcción Lógica de Programas
CMMI: Capability Maturity Model Integration
CRC/RDD (Wirfs-Brock).
CVS: Concurrent Versions System
DFD: Diagrama de Flujo de Datos
DSed: Desarrollo de Sistemas Estructurados en Datos
DSDM: Método de desarrollo de sistemas dinámicos
EssUP: Essential Unified Process
ERS: Especificación de Requisitos Software
FDD: Feature Driven Development
IEEE: Institute of Electrical and Electronics Engineers
JAD: Joint Application Design
JRP: Joint Requirements Planning
JSD: Jackson System Development
JSP: Jackson Structured Programming
LSD: Lean Software Development
OMG: Object Management Group
OMT: Object Modeling Technique
OOAD: Object-Oriented Analysis and Design
OOSE: Object-Oriented Software Engineering
OpenUP: Open Unified Process
PMI: Project Management Institute
PUDS: Proceso Unificado de Desarrollo Software
PVCS: Polytron Version Control System
ROI: Return of Investment o retorno de la inversión
RUP: Rational Unified Process
SCCS: Source Code Control System
TDD: Test Development Driven
UML: Unified Modeling Language
XP: Programación Extrema

RESUMEN ESQUEMÁTICO

- **Análisis Funcional de Sistemas:** Creación de ERS + Modelos de Análisis.
 - **Clasificación** de Requisitos
 - a) Sistema → Hacen referencia a todas las partes implicadas
 - b) Software → Limitados al software a desarrollar
 - i. Funcionales: Especifican la funcionalidad
 - ii. No Funcionales: Seguridad, rendimiento, facilidad de modificación etc.
 - **Características** de los requisitos: Completos, Consistentes, No Ambiguos, Verificables, Comprensibles, Fáciles de Modificar.
 - Actividades del **Establecimiento de Requisitos:**
 - Educación
 - Análisis
 - Representación
 - Validación
 - **Técnicas de Alto Nivel** para Establecimiento de Requisitos
 - Sesiones JAD Y JRP
 - Entorno de Bucles Adaptativo
 - Prototipos
 - Factores Críticos de Éxito
 - Historias de Usuario
 - **Técnicas de Bajo Nivel** para Establecimiento de Requisitos
 - Brainstorming
 - Entrevistas
 - PIECES
 - Casos de Uso
 - Análisis de Mercado
 - **Gestión de Requisitos** → Supone gestionar el cambio
 - Control de versiones
 - Trazabilidad de Requisitos
 - Distintos enfoques Metodologías tradicionales Vs Metodologías Ágiles.
- **Metodologías** → procedimientos, métodos o técnicas, herramientas y un soporte documental
 - Metodologías **Estructuradas**
 - Orientadas a Procesos (DFDs) → DeMarco, Gane&Sarson, Yourdon
 - Orientadas a Datos
 - Orientadas a Datos Jerárquicos → CLP, DSED, Warnier-Orr, JSP, JSD
 - Orientadas a Datos no Jerárquicos → James Martin, Clive Finkelstein
 - Metodologías **Orientadas a Objetos** → OMT, OOAD, OOSE, RUP
 - Metodologías de **Tiempo Real** → Ward y Mellor
 - Metodologías **Ágiles** → 12 principios del Manifiesto Ágil. Metodologías Scrum ASD, AUP, BDD, Crystal Clear, EssUP, FDD, LSD, Kanban, OpenUP, XP, DSDM, TDD, etc.

- **SCRUM**

- Product Backlog: Lista de requisitos
- Burn down Chart: Gráfica de requisitos realizados
- Sprint: Iteraciones (entre 15 y 30 días)
 - Sprint Planning: Reunión de planificación del Sprint
 - Sprint Backlog → Requisitos a implementar en el sprint
 - Sprint Review → Reunión de revisión de Sprint
 - Sprint Retrospective → Para mejora del proceso
- Roles
 - Cerdo → Product Owner (representa al cliente), ScrumMaster (facilitador), ScrumTeam (equipo)
 - Gallina → Usuarios, Stakeholders (interesados), Managers

- **Kanban** → Producción Just-In-Time (JIT). Basado en tarjetas visuales.

- Principios:
 - Visualizar el flujo de trabajo
 - Limitar el trabajo en curso (WIP – Work In Progress)
 - Establecer políticas explícitas de calidad
 - Medir y gestionar el flujo de trabajo
 - Realizar ciclos de retroalimentación
 - Mejorar colaborando y evolucionar experimentando

- **XP** (Extreme Programming): Entregas pequeñas, diseño simple, integración continua.

- Fases:
 1. Exploración
 2. Planificación de la Entrega (Release)
 3. Iteraciones
 4. Producción
 5. Mantenimiento
 6. Finalización (Muerte) del Proyecto
- Valores:
 1. Simplicidad
 2. Comunicación
 3. Retroalimentación
 4. Coraje
 5. Respeto

- **FDD** (Desarrollo basado en funcionalidades):
 - Proceso
 1. Develop an Overall Model (desarrollar modelo general)
 2. Build a Feature List (construir lista de Funcionalidades)
 3. Plan by Feature (planificar en función de las Funcionalidades)
 4. Iteración: Design and Build by Feature
- **TDD** (Test Driven Development) -> Escribir las pruebas primero +Refactorización.
 - Ciclo de desarrollo:
 1. Elegir un requisito
 2. Escribir una prueba
 3. Verificar que la prueba falla
 4. Escribir la implementación
 5. Ejecutar las pruebas automatizadas
 6. Eliminación de duplicación
 7. Actualización de la lista de requisitos
- **Escalado Ágil** → Enfocado a procesos ágiles a nivel organización. Centrado en coordinación de equipos ágiles.
 - Frameworks principales:
 - Nexus
 - LeSS
 - SAFe

PREGUNTAS DE TEST

- 1) **El Manifiesto Ágil recoge los principios de las metodologías de desarrollo ligeras o ágiles. Indique cuál de las siguientes frases forma parte del manifiesto:**
 - a) Entregamos software funcional frecuentemente, entre dos semanas y dos meses
 - b) No esperamos por requisitos, añadimos la máxima funcionalidad antes de conocerlos.
 - c) Los usuarios componen los requisitos, dejándolos cerrados en la primera etapa de desarrollo.
 - d) El equipo de trabajo ágil debe saber trabajar bajo presión y abrazar la irregularidad.
- 2) **¿Cuál de las siguientes no es una metodología ágil de desarrollo?**
 - a) Programación Extrema
 - b) SCRUM
 - c) RDS
 - d) Feature Driven Development (FDD)
- 3) **En el desarrollo de software ágil (Agile) se usan los siguientes niveles de planificación:**
 - a) Planificación de entregas, planificación de iteraciones o sprints y planificación diaria.
 - b) Planificación estratégica y planificación semanal.
 - c) Planificación de iteraciones o sprints, planificación de hitos de cada iteración o sprint y planificación semanal.
 - d) Planificación estratégica, planificación de recursos y planificación de entregas.
- 4) **En SCRUM se consideran las siguientes tipos de reuniones**
 - a) Daily Scrum, Sprint Review Meeting, Sprint Retrospective, Scrum de Scrum, Sprint Planning Meeting
 - b) Project Scrum, Scrum review, Sprint Planning Meeting, Sprint Review Meeting, Sprint Retrospective
 - c) Daily Scrum, Scrum de Scrum, Sprint Planning Meeting, Sprint Retrospective
 - d) Ninguna de las anteriores es correcta
- 5) **En relación al product backlog de Scrum señale la afirmación correcta**
 - a) Es un documento detallado donde se describe cómo se van a implementar los requisitos.
 - b) Las tareas se dividen en horas con ninguna tarea de duración superior a 16 horas.
 - c) Contiene descripciones genéricas de todos los requerimientos y funcionalidades deseables priorizadas.
 - d) Todas las anteriores son correctas
- 6) **Señale cuál de las siguientes afirmaciones no se corresponde a un Principio del Manifiesto Ágil**
 - a) La atención continua a la excelencia técnica enaltece la agilidad.
 - b) La simplicidad como arte de maximizar la cantidad de trabajo que no se hace, es esencial.
 - c) Las mejores arquitecturas, requisitos y diseños emergen de equipos que se auto-organizan.
 - d) Todas las afirmaciones anteriores se corresponde a un Principio del Manifiesto Ágil
- 7) **SCRUM es una metodología:**
 - a) Clásica.
 - b) Extrema.
 - c) Orientada a Objetos.
 - d) Ninguna de las anteriores.

8) Las Metodologías Estructuradas se clasifican en:

- a) Orientadas a Procesos
- b) Orientadas a Datos Jerárquicos
- c) Orientadas a Datos no Jerárquicos
- d) Todas las anteriores

9) En relación a los tipos de Herramientas CASE, señale la que no lo sea:

- a) Herramientas de Diagramación para la creación de diagramas y especificaciones gráficas.
- b) Generadores de entrevistas para la obtención de requisitos.
- c) Generadores de Pantallas e Informes (prototipado de la interfaz de usuario, repositorios, etc.)
- d) Análisis, Diseño y Verificación

10) En relación a las clases de Herramientas CASE por la fase que soportan, señale la que no lo sea:

- a) Upper CASE o CASE Superior
- b) Middle CASE o CASE Intermedio
- c) Super CASE o CASE Superior
- d) Lower CASE o CASE Inferior

11) En relación a los tipos de Herramientas CASE, señale la que lo sea:

- a) Generadores de Código
- b) Mantenimiento
- c) Planificación y Gestión de Proyectos
- d) Todas las anteriores

12) El proceso unificado de desarrollo de software está dirigido por «casos de uso». Un «caso de uso» es:

- a) Un diagrama que muestra el despliegue de un sistema desde un punto de vista estático.
- b) Un fragmento de funcionalidad del sistema, que proporciona al usuario un resultado importante.
- c) Un conjunto de objetos y relaciones, incluyendo los mensajes que pueden enviarse entre ellos.
- d) Un diagrama que muestra el flujo de actividades

13) Señale de los siguientes conceptos cuál se corresponde con una metodología de desarrollo orientada a objetos:

- a) DeMarco.
- b) RUP.
- c) XP.
- d) Warnier-Orr.

14) La técnica de Desarrollo Conjunto de Aplicaciones (JAD, Joint Application Development):

- a) Se aplica para identificar requisitos en el campo de los sistemas de información.
- b) Implica más a los ingenieros de requisitos y menos a los usuarios y clientes.
- c) Requiere más tiempo para contrastar las opiniones con los clientes.
- d) Implica la realización de entrevistas individuales para su posterior puesta en común.

15) ¿Cuál de las siguientes es una herramienta que ayuda a realizar seguimiento a los requisitos a lo largo del ciclo de vida del proyecto para asegurar que se están cumpliendo de manera eficaz?

- a) Cuestionarios.
- b) Casos de uso.
- c) Matriz de trazabilidad de requisitos.
- d) Prototipos.

16) En la PROGRAMACIÓN EXTREMA (XP) se persigue:

- a) Reducir el número de entregas del sistema.
- b) Una alta relación con el usuario.
- c) Eliminar las pruebas de integración.
- d) Eliminar las pruebas unitarias.

17) ¿Cuál de las siguientes es una característica propia del Proceso Unificado de Desarrollo de Software (PUDS)?

- a) Se basa en las Historias de Usuario.
- b) Proceso centrado en la arquitectura.
- c) Existe una continua refactorización del código.
- d) No contempla la gestión del proyecto

18) ¿A cuál de las siguientes opciones corresponde la definición siguiente?: «Un medio a través del cual se simula el aspecto visual del sistema mediante la representación de los conceptos, componentes, objetos gráficos, entradas y salida requeridas para la ejecución de cada función en respuesta a las necesidades planteadas»

- a) Diseño orientado a objetos.
- b) JRP.
- c) Prototipado.
- d) JAD

19) ¿Cuál de las siguientes es una metodología ágil de desarrollo?

- a) AJAX
- b) ASD
- c) SOFTAG
- d) FDR

20) ¿Cuál de los siguientes contradice un principio de las metodologías ágiles de desarrollo software (principios del Manifiesto Ágil)?

- a) Son bienvenidos los requisitos cambiantes, incluso si llegan en las etapas tardías del desarrollo.
- b) Los responsables de negocio y los desarrolladores trabajan juntos de forma cotidiana durante todo el proyecto.
- c) El software, en su fase de diseño, es la medida principal de progreso.
- d) La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	A
2	C
3	A
4	A
5	C
6	D
7	D
8	D
9	B
10	C
11	D
12	B
13	B
14	A
15	C
16	B
17	B
18	C
19	B
20	C

BIBLIOGRAFÍA BÁSICA

1. <https://www.agilealliance.org/>
2. Mike Cohn: Agile Estimating and Planning, 2006, Prentice Hall
3. Mike Cohn, "User Stories Applied", 2004, Addison Wesley
4. Beck.K, Beedle, M. et al. Manifiesto for Agile Software Development. 2001
<http://AgileManifesto.org/>
5. <http://www.proyectosagiles.org/>
6. Metodología Métrica versión 3. Ministerio de Administraciones Públicas. 1998.
http://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html
7. Pressman, Roger. Ingeniería del Software. Un Enfoque Práctico. Ed. McGraw Hill. 6ª Ed. 2005. ISBN 970-105473-3
8. Sommerville, Ian. Ingeniería del Software. Ed. Addison Wesley. 7ª Ed. 2005. ISBN: 9788478290741
9. Gonzalo Cuevas, Agustín. Gestión del Proceso de Software. Ed. Centro de Estudios Ramón Areces. 1ª Ed. ISBN 84-8004-546-9
10. Pressman, Roger. Ingeniería del Software. Un Enfoque Práctico. Ed. McGraw Hill. 6ª Ed. ISBN 970-105473-3
11. Sommerville, Ian. Ingeniería del Software. Ed. Pearson-Addison-Wesley. 7ª Ed. 2005. ISBN 84-7829-074-5
12. Thayer, RH y Dorfman, M. Software Requirements Engineering. 2ª Ed, IEEE Computer Society Press, 1997.

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

13. <http://www.omg.org/spec/UML/>
14. <http://agilemodeling.com/>
15. Dorfman and R.H.Thayer. Software Engineering. IEEE Computer Society Press. 1997
16. Whitten et. al. Análisis de Sistemas y Métodos de Diseño. Ed. Irwin. 3ª Ed. 1996.
17. <http://www.scaledagileframework.com/>
18. <https://less.works/>
19. <https://www.scrum.org/resources/nexus-guide>